OmniSensus Medical

AN INTERNSHIP REPORT

Submitted by

Parth Vadodariya

(230043107015)

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

In

COMPUTER ENGINEERING

B. H. GARDI COLLEGE OF ENGINEERING & TECHNOLOGY

RAJKOT

Gujarat Technological University, Ahmedabad

[January, 2026 – June, 2026]

B.H. GARDI COLLEGE OF ENGINEERING AND TECHNOLOGY

Rajkot, Kalawad Highway, Anandpar, Gujarat 361162

CERTIFICATE

This is to certify that the internship report submitted along with the Project entitled **OmniSensus Medical** has been carried out by **Parth Vadodariya** under my guidance in partial fulfilment for the degree of Bachelor of Engineering in **COMPUTER ENGINEERING**, 8th Semester of Gujarat Technological University, Ahmedabad during the academic year 2026-27.

Prof. Harsh Mehta — Internal Guide

Prof. Monika R. Shah — Head of the Department

COMPANY CERTIFICATE

4/7/26, 4:20 AM Gardi Vidyapith Mail - Re-Confirmation of Selection: MS Elevate - GTU Internship Program

Parth Vadodariya <22ce010@gardividyapith.ac.in>

Re-Confirmation of Selection: MS Elevate - GTU Internship Program

2 messages

Edunet Foundation <anjali@edunetfoundation.org> Fri, Jan 9, 2026 at 6:39 PM
Reply-To: anjali@edunetfoundation.org
To: 22ce010@gardividyapith.ac.in

Dear Students,

Greetings!

Thank you for your interest in the Microsoft Elevate – GTU Internship Program. We appreciate the overwhelming response and the time and effort you invested in submitting your application.

After a careful review of all applications in accordance with the program's eligibility criteria, we are pleased to inform you that your application has been successfully selected for the **Microsoft Elevate – GTU Internship Program.**

Congratulations on your selection!

Please note the following updates:

• In the coming week, you will receive an official Telegram group link to join.

• Once the group is created, **all further communication and updates will be shared exclusively through the Telegram group**.

• An online orientation session will be conducted shortly after the group is formed. Details regarding the date, time, and agenda of the orientation will be shared in the group.

We request you to wait for the Telegram group link and avoid multiple follow-ups in the meantime. This will help us ensure smooth and timely communication with everyone.

We look forward to your active participation and wish you a rewarding learning experience.

Warm regards,

Team FICE

COMPLETION CERTIFICATE

//It will be replace with completion certificate so ignore it

Parth Vadodariya <22ce010@gardividyapith.ac.in>

Re-Confirmation of Selection: MS Elevate - GTU Internship Program

2 messages

Edunet Foundation <anjali@edunetfoundation.org> Fri, Jan 9, 2026 at 6:39 PM
Reply-To: anjali@edunetfoundation.org
To: 22ce010@gardividyapith.ac.in

Dear Students,

Greetings!

Thank you for your interest in the Microsoft Elevate – GTU Internship Program. We appreciate the overwhelming response and the time and effort you invested in submitting your application.

After a careful review of all applications in accordance with the program's eligibility criteria, we are pleased to inform you that your application has been successfully selected for the **Microsoft Elevate – GTU Internship Program.**

Congratulations on your selection!

Please note the following updates:

• In the coming week, you will receive an official Telegram group link to join.

• Once the group is created, **all further communication and updates will be shared exclusively through the Telegram group**.

• An online orientation session will be conducted shortly after the group is formed. Details regarding the date, time, and agenda of the orientation will be shared in the group.

We request you to wait for the Telegram group link and avoid multiple follow-ups in the meantime. This will help us ensure smooth and timely communication with everyone.

We look forward to your active participation and wish you a rewarding learning experience.

Warm regards,

Team FICE

DECLARATION

We hereby declare that the Internship report submitted along with the Internship and project entitled OmniSensus Medical submitted in partial fulfilment for the degree of Bachelor of engineering in Computer Engineering to Gujarat Technological University, Ahmedabad, is a bonafide record of original work carried out by me at Microsoft Elevate FICE under the supervision of Mr. Harsh Mehta Amitbhai and that no part of this report has been directly copied from any students' report or taken from any other source, without providing due reference.

Name of the Student Sign of Student

Parth Vadodariya

ACKNOWLEDGEMENT

I am deeply grateful to **Microsoft Elevate** for providing me the opportunity to undertake this internship in the domain of full-stack AI-powered web application development. The experience has been immensely enriching, both technically and professionally.

I would like to express my sincere gratitude to my Internal Guide, **Prof. Harsh Mehta**, for his constant support, insightful feedback, and encouragement throughout the duration of this project. His guidance was instrumental in shaping the **OmniSensus Medical** platform into a structured and high-quality deliverable.

I also extend my heartfelt thanks to **Prof. Monika R. Shah**, Head of the Computer Engineering Department, B. H. Gardi College of Engineering & Technology, Rajkot, for her academic support and motivating environment. I am thankful to all the faculty members of the Computer Engineering department who provided valuable inputs during internal reviews.

Lastly, I am grateful to Gujarat Technological University, Ahmedabad, for framing such a practical and industry-oriented curriculum that gave me the opportunity to apply theoretical knowledge to a real-world healthcare problem.

Parth Vadodariya (230043107015)

ABSTRACT

OmniSensus Medical addresses the healthcare challenge of delayed diagnosis and fragmented patient data. In real-world clinical settings, manual data analysis often slows critical interventions. This project provides a unified intelligence platform that centralizes complex biomarkers, functioning as a decision-support tool that automatically converts raw clinical data into professional health reports. This reduces administrative burdens on medical staff and minimizes predictive uncertainty. By integrating an intelligent assistant to interpret system-wide data, the platform ensures high-risk cases are identified early. Ultimately, OmniSensus Medical optimizes hospital resources and improves patient outcomes through rapid, acces resources and improves patient outcomes through rapid, accessible, and automated sible, and automated preventive care.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

- **AI** – Artificial Intelligence
- **API** – Application Programming Interface
- **AUC** – Area Under the Curve
- **CKD**– Chronic Kidney Disease
- **CORS** – Cross-Origin Resource Sharing
- **CSS** – Cascading Style Sheets
- **CVD** – Cardiovascular Disease
- **DB** – Database
- **DXA** – Dual-energy X-ray Absorptiometry (twip unit in docx)
- **eGFR** – Estimated Glomerular Filtration Rate
- **HbA1c** – Glycated Haemoglobin
- **HTML** – HyperText Markup Language
- **HTTP** – HyperText Transfer Protocol
- **JWT** – JSON Web Token
- **KPI** – Key Performance Indicator
- **LDL** – Low-Density Lipoprotein
- **ML** – Machine Learning
- **MVP** – Minimum Viable Product
- **ORM** – Object-Relational Mapping
- **PDF** – Portable Document Format
- **REST** – Representational State Transfer
- **ROC** – Receiver Operating Characteristic
- **SHAP** – SHapley Additive exPlanations
- **SQL** – Structured Query Language
- **SSL** – Secure Sockets Layer
- **UI/UX** – User Interface / User Experience
- **UUID** – Universally Unique Identifier

CHAPTER 1. INTRODUCTION

1.1 History / Company Overview

Figure 1.1 : Company Logo

Address : Microsoft, Microsoft Corporation (India) Pvt Ltd., Prestige Galaxy Ferns, Bellandur Outer Ring Road, Bengaluru - 560103

Mobile : +91 8171215655

Mail : anjali@fice.in

Industry : Education Technology (EdTech) / Professional Skill Development.

Founded : 2023

- Microsoft Elevate is a technology skill-development and innovation initiative by Microsoft, designed to empower engineering students and early-career professionals with hands-on experience in cloud computing, artificial intelligence, and full-stack software development.
- As part of this initiative, students are placed in structured virtual project environments where they develop scalable, production-ready software systems. The project OmniSensus Medical was developed under this programme, giving exposure to multi-tier web architecture, machine learning model integration, RESTful API development, relational database design, and frontend engineering using modern JavaScript frameworks.
- The internship period ran from January 2026 to April 2026, spanning 12 weeks of structured development. The technical environment included Microsoft Azure for cloud resources, GitHub for version control, and industry-standard tools such as VS Code, Postman, and Neon PostgreSQL.

1.2 Products and Scope of Work

The primary product developed during this internship is OmniSensus Medical — a full-stack, AI-powered clinical intelligence platform. The scope of work encompassed all three tiers of a modern web application:

- **Backend REST API**: FastAPI (Python 3.11) with 60+ endpoints, JWT authentication, role-based access control, and PostgreSQL integration via asyncpg.
- **ML Inference Service**: Flask microservice hosting three Random Forest models calibrated with CalibratedClassifierCV, serving diagnostic predictions, SHAP explanations, and PDF report generation.
- **Frontend SPA**: Next.js 14 with App Router, three role-specific dashboard systems (Admin, Doctor, Patient), and a 27-file CSS design system.
- **Database Design**: 27-table PostgreSQL schema with 8 views, 12 triggers, and comprehensive seed data spanning 67 diagnostic runs across 16 patients.
- **DevOps:** Deployment on Render free tier with Neon serverless PostgreSQL.

1.3 Portfolio

The **OmniSensus Medical** platform represents a comprehensive demonstration of full-stack engineering capability. Key deliverables include:

- A production-grade REST API with complete CRUD for patients, doctors, diagnostics, appointments, medications, and notifications.
- Three calibrated ML models with domain-specific biomarker input pipelines, producing composite health scores, SHAP-ranked clinical insights, and triage urgency levels.
- A responsive Next.js frontend with three complete role dashboards, 15+ unique page components, Chart.js data visualisations (radar, line, bar, doughnut), and a PharmaBot chat interface.
- A 237 KB master seed SQL file populating all 31 database tables with realistic anonymised clinical data using a TRUNCATE-then-INSERT strategy.
- Complete project documentation including ER diagrams, API reference, ML model specifications, and deployment guides.

CHAPTER 2. OVERVIEW OF DEPARTMENT

2.1 Full-Stack Web Development

This department focused on the design and development of the web application layer. Activities included:

- Frontend Development: Building the Next.js 14 application with the App Router, implementing server-side and client-side rendering strategies, managing authentication state, and developing role-specific layouts for Admin, Doctor, and Patient roles.
- Backend API Development: Designing and implementing 60+ RESTful endpoints in FastAPI, including JWT token-based authentication, refresh token rotation, role-based middleware, and comprehensive error handling.
- Database Engineering: Designing the PostgreSQL schema across two migration versions (27 tables, 8 views, 12 triggers), writing optimised queries, and creating database views for efficient frontend data consumption.
- CSS Design System: Building a 27-file modular CSS system using CSS custom properties (variables) for theming, with distinct styling for each role dashboard.

2.2 Machine Learning and Artificial Intelligence

This department handled the design, training, evaluation, and deployment of the ML inference models. Activities included:

- Model Training: Training three binary classification models — Heart Disease, Diabetes (PIMA dataset), and Chronic Kidney Disease — using scikit-learn's RandomForestClassifier wrapped in CalibratedClassifierCV for well-calibrated probability outputs.
- Model Evaluation: Computing ROC-AUC scores (Heart: 91.3%, Diabetes: 82.1%, Kidney: 100%), confusion matrices, and classification reports for each model.
- Composite Scoring: Designing the weighted composite health score formula combining the three domain risks with empirically tuned weights (Heart: 0.40, Diabetes: 0.35, Kidney: 0.25).
- Explainability: Implementing SHAP (SHapley Additive exPlanations) to produce feature importance rankings for each diagnostic run, stored in the ai_insights table.
- Flask Microservice: Building the inference API in Flask with endpoints for diagnosis, triage assessment, readmission risk, drug recommendation, PharmaBot chat, and PDF report generation.

CHAPTER 3. INTERNSHIP AND PROJECT

3.1 Internship Summary

The internship was completed as part of the Microsoft Elevate programme, focusing on full-stack AI-powered web application development in the healthcare domain. Over 12 weeks (January 2026 to April 2026), the project OmniSensus Medical was designed, developed, tested, and deployed, covering the complete software development lifecycle from requirements gathering to production deployment.

3.2 Purpose

The purpose of this internship was to gain comprehensive, industry-grade experience in building a production-quality, multi-tier web application that solves a real-world healthcare problem. The aim was to develop deep understanding of full-stack web application architecture, machine learning model integration with web APIs, role-based multi-user system design, database schema design for complex healthcare data relationships, and REST API design principles, security, and documentation.

3.3 Objective

The primary objectives of the internship and the OmniSensus Medical project were:

1. To design and implement an AI-powered clinical decision support system that automatically stratifies patient risk into Stable, Borderline, and Critical tiers.
2. To build a secure, role-based web platform serving System Administrators, Doctors, and Patients with dedicated dashboards and workflows.
3. To integrate three machine learning models for cardiovascular, metabolic, and renal disease risk assessment, producing calibrated probability estimates and SHAP-ranked clinical insights.
4. To implement a comprehensive PostgreSQL database schema with 27 tables, 8 views, and automated triggers for data consistency.
5. To develop a Flask-based ML microservice exposing diagnostic inference, triage assessment, drug recommendation, and PDF report generation endpoints.
6. To deploy the complete platform on cloud infrastructure and produce thorough technical documentation.

3.4 Scope

The scope of OmniSensus Medical covers user management with JWT token security, clinical diagnostics with three ML models, longitudinal patient management, appointment workflows, institutional administration, PharmaBot AI assistance, automated PDF report generation, role-specific notification systems, and a 27-table database with 8 analytical views and 12 automated triggers.

3.5 Introduction to Project

OmniSensus Medical is an automated clinical decision support and integrated health intelligence platform. It addresses the fragmented and manual nature of clinical data analysis in hospital settings by providing a centralised, AI-powered intelligence layer over patient biomarker data.

At its core, the platform accepts structured patient vitals and laboratory results through a guided 3-step diagnostic wizard, passes them through three calibrated ML classifiers, and produces a composite health score (0–100) with domain-level risk breakdowns (cardiovascular, metabolic, renal). A SHAP-based explainability engine identifies the top biomarkers driving each patient's risk, enabling clinicians to make informed, data-backed clinical decisions.

3.6 Project Purpose

The purpose of OmniSensus Medical is to eliminate the diagnostic delay and data fragmentation that characterises traditional paper-based or siloed digital health records systems. The platform automates conversion of raw biomarker data into structured risk scores, automatically flags Critical or Borderline patients for immediate clinical attention, reduces administrative burdens through automated workflows and triggers, provides patients with transparent access to their health data, and gives hospital administrators real-time visibility into platform-wide health trends.

3.7 Project Objectives

7. Implement three machine learning classifiers (Heart, Diabetes, Kidney) with AUC scores above 80% using calibrated Random Forest models.
8. Build a composite health scoring system that combines domain risks with physiologically motivated weights to produce a single patient wellness score.
9. Design a 27-table PostgreSQL schema with appropriate foreign keys, constraints, views, and triggers to maintain data integrity automatically.
10. Develop a FastAPI backend with 60+ REST endpoints covering full CRUD for all entities, with JWT authentication and role-based access control enforced at every endpoint.
11. Build a Next.js 14 frontend with three role-specific dashboards, minimum 15 unique page components, and Chart.js visualisations for health trend analysis.
12. Generate SHAP feature importance rankings for every diagnostic run and store them in the database for clinical transparency.
13. Implement automated medication adherence tracking with dose-level precision.
14. Produce PDF diagnostic reports via ReportLab for every completed diagnostic run.
15. Deploy the complete platform on cloud infrastructure with a functional, publicly accessible URL.

3.8 Project Scope

- Authentication and Authorisation: JWT Bearer tokens, refresh token rotation, bcrypt password hashing, role-based middleware enforcement.
- Patient Module: Full patient profile management, longitudinal diagnostic run history, vitals tracking, medication adherence monitoring, appointment scheduling, notifications, and health report archive.
- Doctor Module: Patient panel, diagnostic wizard (3-step biomarker input), AI score results with domain breakdown and SHAP insights, appointment management, patient lab trend charts.
- Admin Module: Platform KPI dashboard, triage monitoring queue, user registry management, audit log viewer, bed occupancy heatmap, equipment status tracker, on-call schedule manager, ML model performance analytics.
- ML Diagnostic Engine: Three calibrated classifiers, composite scoring formula, triage urgency engine, SHAP explainability, readmission risk model, drug recommendation engine.
- PharmaBot Assistant: Conversational AI assistant with persistent session tracking, available to all three roles for health-related queries.

3.9 Technology and Tools

3.9.1 Technology Stack

Frontend:

- Next.js 14 (App Router): React 18-based full-stack framework providing server-side rendering, API routes, and a file-based routing system.
- CSS Custom Properties: 27-file modular design system with CSS variables for theming, role-specific colour schemes, and responsive layouts.
- Chart.js 4.4: Data visualisation library used for radar charts, line charts, bar charts, and doughnut charts.

Backend API:

- FastAPI (Python 3.11): High-performance async REST framework providing automatic OpenAPI documentation, Pydantic validation, and dependency injection.
- SQLAlchemy / asyncpg: Async PostgreSQL driver for high-throughput database operations.
- JWT (python-jose): JSON Web Token authentication with access and refresh token support.
- bcrypt ($2b$12): Password hashing with a cost factor of 12 for strong security.

ML Inference Service:

- Flask (Python 3.11): Lightweight WSGI microframework serving the ML inference API on a separate port.
- scikit-learn: CalibratedClassifierCV(RandomForest) for all three disease domain classifiers.
- SHAP: SHapley Additive exPlanations for model interpretability and feature importance ranking.
- ReportLab: PDF generation library for automated clinical diagnostic reports.

Database:

- PostgreSQL 16 (Neon Serverless): Cloud-native serverless PostgreSQL with connection pooling, SSL enforcement, and branching support.

3.9.2 Tools

- Visual Studio Code: Primary IDE with Python, JavaScript, and PostgreSQL extensions.
- Postman: REST API testing and endpoint validation across all 60+ endpoints.
- Git and GitHub: Version control and collaboration with feature branch workflow.
- Render Dashboard: Cloud deployment management for FastAPI and Flask services.

3.10 Internship Scheduling

Week	Topics / Milestones
Week 1	Requirement analysis, system architecture design, technology selection
Week 2	PostgreSQL schema design (22 core tables), ER diagram
Week 3	FastAPI project setup, auth module (login, JWT, refresh, logout)
Week 4	Patient, Doctor, Admin API routes – CRUD endpoints
Week 5	ML model training (Heart, Diabetes, Kidney), calibration, evaluation
Week 6	Flask ML service – inference endpoints, SHAP integration
Week 7	Next.js project setup, CSS design system, auth page, layouts
Week 8	Admin dashboard, analytics, triage monitor, audit log
Week 9	Doctor dashboard, diagnostic wizard, patient panel, AI Lab
Week 10	Patient dashboard, reports, medications, appointments pages
Week 11	v2 migration, new tables, PharmaBot, PDF reports, notifications
Week 12	Testing, seed data, deployment, documentation, final review

Table 3.10.1: Internship Scheduling Plan

CHAPTER 4. SYSTEM ANALYSIS

4.1 Study of Current System

In traditional clinical settings, healthcare data management suffers from several systemic deficiencies that this project aims to address:

- Diagnostic Data Fragmentation: Patient biomarker data is typically spread across paper lab reports, disconnected EMR modules, and manual spreadsheets. There is no unified view of a patient's multi-domain health status.
- Manual Risk Assessment: Clinical risk stratification is performed manually by physicians, which is time-consuming, subjective, and prone to human error. There is no automated system that combines cardiovascular, metabolic, and renal risk domains into a single unified score.
- Delayed Intervention: Without real-time risk scoring and alerting, high-risk patients may not be identified promptly, leading to delayed clinical intervention and poorer health outcomes.
- No Longitudinal Trend Analysis: Doctors often lack tools to visualise a patient's health score trajectory over time, making it difficult to assess whether clinical interventions are producing measurable improvements.
- Administrative Overhead: Managing appointments, medication prescriptions, bed allocation, and staff scheduling across departments requires significant manual effort with no automated workflow support.
- No Patient Engagement Layer: Patients typically have no access to their own clinical data, diagnostic results, or medication schedules, reducing adherence and engagement with their own health management.

4.2 Requirements of New System

- **Automated AI Diagnosis:** Automatically process patient biomarker inputs through ML models and produce domain-specific risk scores, composite health scores, and clinical flags without manual analysis.
- **Role-Based Access**: Provide distinct, purpose-specific interfaces for System Administrators, Doctors, and Patients, each with appropriate data access and functionality.
- **Real-Time Risk Alerting**: Automatically generate critical-priority notifications when a patient is classified as Critical tier, ensuring timely clinical response.
- **Longitudinal Health Trends**: Store every diagnostic run with full biomarker data and visualise health score trends over time through interactive charts.
- **Medication Management**: Track prescribed medications, monitor dose-level adherence, and display adherence percentages to both doctors and patients.
- **Appointment Workflow**: Enable appointment booking, confirmation, completion, and cancellation with automated reminder notifications.
- **Explainable AI**: Every AI-generated risk score must be accompanied by SHAP feature importance rankings, giving clinicians transparent, interpretable reasoning for the model's output.

4.3 Proposed System

4.3.1 Frontend (Next.js 14)

A responsive single-page application with role-based routing, server-side rendering for initial page loads, and client-side interactivity for dynamic dashboard components. Three complete role layouts (Admin, Doctor, Patient) each with dedicated sidebars, notification badge counts, and role-specific navigation menus.

4.3.2 Backend API (FastAPI)

A high-performance async REST API serving as the orchestration layer between the frontend, database, and ML service. JWT Bearer authentication with 30-minute access token expiry, role-based dependency injection enforced at every endpoint, ML proxy endpoints that forward diagnostic requests to the Flask service and write results back to PostgreSQL.

4.3.3 ML Inference Service (Flask)

A separate Flask microservice hosting three calibrated Random Forest models, ensuring ML workloads do not block web request processing. Key endpoints include POST /diagnostics for composite risk assessment, POST /report/generate for PDF generation, POST /chat for PharmaBot, and POST /triage for standalone urgency computation.

4.3.4 Database (PostgreSQL 16 / Neon)

A 27-table relational schema designed for clinical data integrity. Cascade delete relationships ensure no orphaned records. Automated triggers sync patient health scores after every diagnostic run. Eight analytical views provide pre-computed JOIN queries for patient summaries, triage queues, platform analytics, and model performance.

4.4 System Feasibility

4.4.1 Technical Feasibility

OmniSensus Medical uses well-established, production-proven open-source technologies (FastAPI, Next.js, PostgreSQL, scikit-learn, Flask). All components are available without licensing costs. The Neon PostgreSQL serverless platform provides auto-scaling database infrastructure. All technologies have extensive documentation, community support, and proven deployments at scale.

4.4.2 Economic Feasibility

The complete technology stack uses open-source frameworks with no licensing fees. Deployment uses free-tier cloud services (Render, Neon, Netlify). The total infrastructure cost for the demonstration deployment is zero, with a clear upgrade path to paid tiers for production-scale usage.

4.4.3 Operational Feasibility

The system is designed with intuitive role-specific interfaces requiring minimal training. The 3-step diagnostic wizard guides doctors through biomarker input sequentially, reducing input errors. Automated notifications, triggers, and background processes reduce manual operational overhead.

4.5 Hardware and Software Requirements

Software Requirements:

Category	Requirement
Operating System	Windows 10/11, macOS 12+, Ubuntu 20.04+ (server)
Browser	Google Chrome 100+, Mozilla Firefox 100+, Microsoft Edge 100+
Backend Runtime	Python 3.11+
Frontend Runtime	Node.js 18+ (LTS)
Database	PostgreSQL 16 (Neon Serverless)
Backend Framework	FastAPI 0.104+, Flask 3.0+
Frontend Framework	Next.js 14+, React 18+
ML Libraries	scikit-learn 1.3+, shap 0.43+, numpy 1.26+, pandas 2.1+
Editor	Visual Studio Code (recommended)
API Testing	Postman 10+
Version Control	Git 2.40+, GitHub

Table 4.5.1: Software Requirements

Hardware Requirements:

Component	Minimum Specification
Processor	AMDA RYZEN 3 or equivalent
RAM	8 GB (16 GB recommended for ML model training)
Storage	20 GB free disk space
Network	Broadband internet (minimum 10 Mbps for Neon database connectivity)
Display	1280 × 800 resolution minimum

Table 4.5.2: Hardware Requirements

CHAPTER 5. SYSTEM DESIGN

5.1 System Design and Methodology

OmniSensus Medical follows a microservices architecture with three independently deployable tiers. The three-tier separation ensures the ML workload (computationally intensive model inference, SHAP computation, PDF generation) does not block web API request processing. The FastAPI backend acts as the orchestrator, forwarding diagnostic requests to Flask, receiving results, and persisting structured data to PostgreSQL.

The design is role-driven: every database query, every API endpoint, and every UI component is scoped to the authenticated user's role. The JWT payload carries the user_id, username, and role, which are decoded at the backend middleware layer and used to enforce data isolation between roles.

5.2 Use Case Diagram

The use case diagram below illustrates the primary interactions between the three user roles (Doctor, Patient, Administrator) and the OmniSensus Medical platform. It shows the key use cases within the Authentication module, Doctor Portal, Patient Portal, and ML System, along with their include relationships.

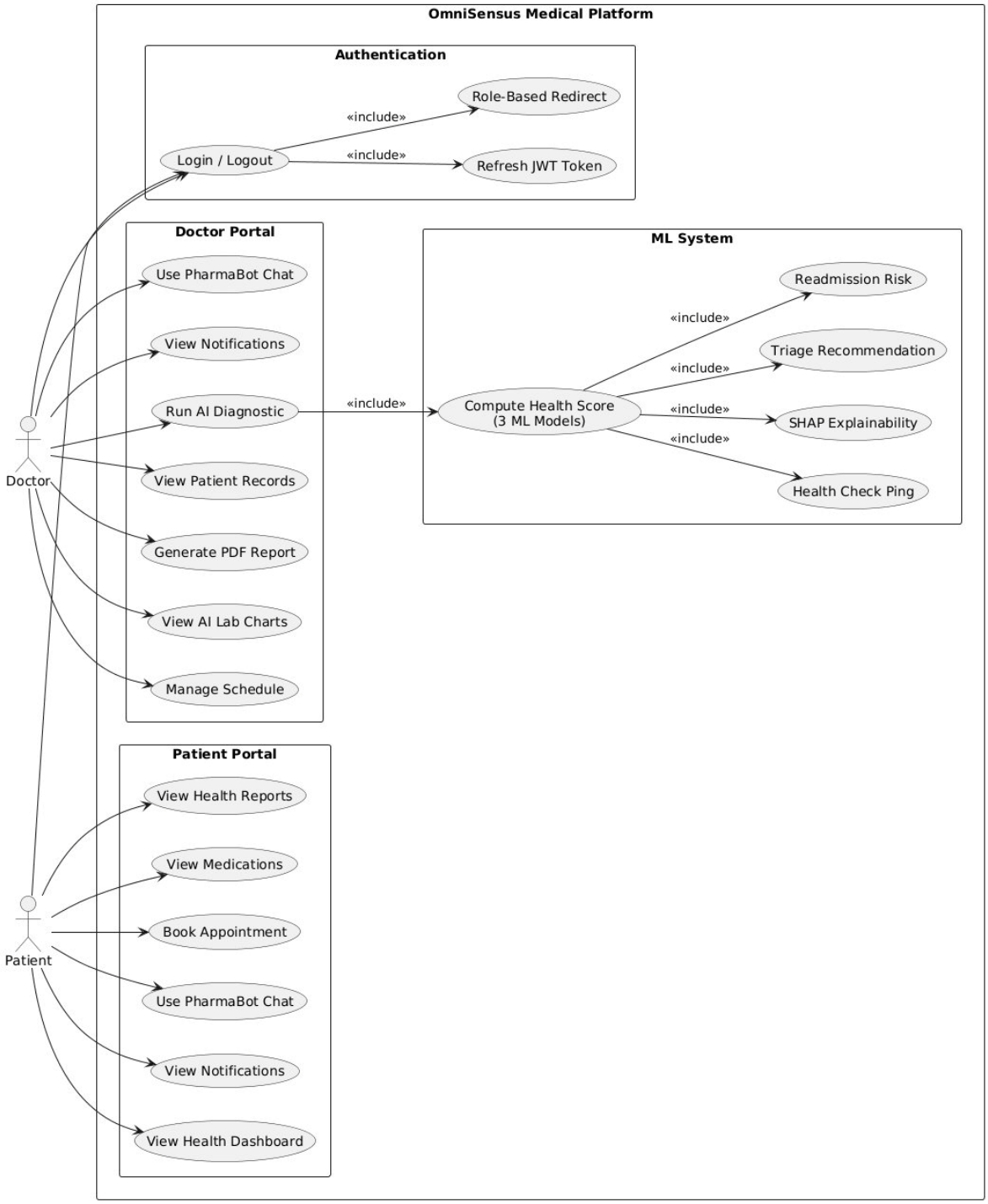


Fig 5.2 : Use Case Diagram – OmniSensus Medical Platform

5.3 Activity Diagrams

5.3.1 Doctor Workflow Activity Diagram

The activity diagram below traces the complete doctor workflow all available clinical actions including diagnostic wizard execution, AI Lab biomarker visualisation, PharmaBot consultation, patient panel browsing, and schedule management.

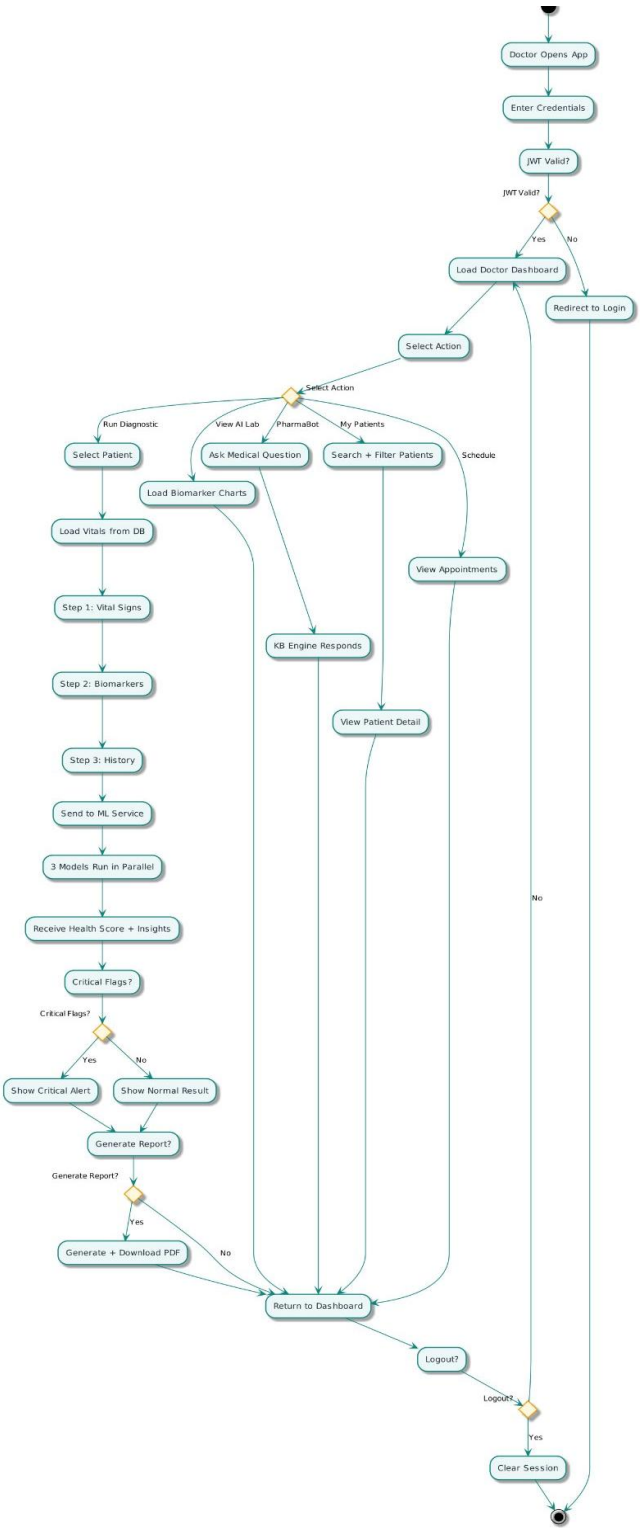


Fig 5.3 : Activity Diagram – Doctor Workflow

5.3.2 Patient Journey Activity Diagram

The patient journey activity diagram traces the patient's interaction with the platform from login through health dashboard rendering, tab navigation (Health Overview, My Reports, Medications, Appointments, PharmaBot), and all downstream actions including report preview, PDF download, medication viewing, appointment booking, and health query submission. The risk tier decision node (Critical / Borderline / Stable) determines the colour-coded health status displayed.

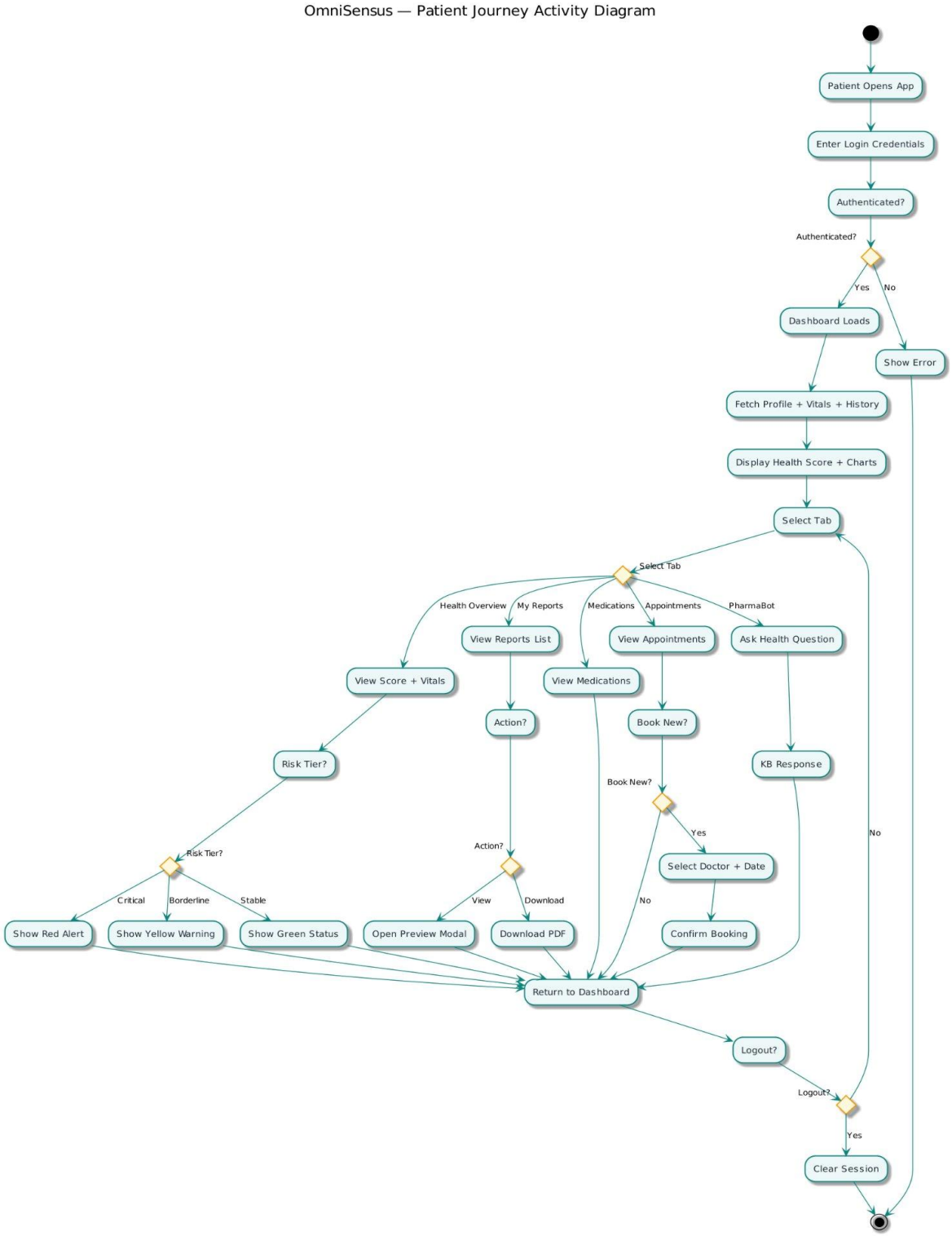


Fig 5.4 : Activity Diagram – Patient Journey

5.4 Sequence Diagrams

5.4.1 AI Diagnostic Run Flow

The sequence diagram below shows the complete message flow for a diagnostic run — from the doctor filling the 3-step wizard in the Next.js frontend, through the FastAPI backend JWT validation and patient context retrieval, to the Flask ML service running three classifiers in parallel, computing the composite health score via OmniScorer, generating SHAP feature importances via OmniExplainer, routing triage urgency via OmniTriage, forecasting trend via OmniAnalytics (ARIMA), and computing readmission risk via OmniReadmission. The complete result is persisted to PostgreSQL and returned to the frontend for animated rendering.

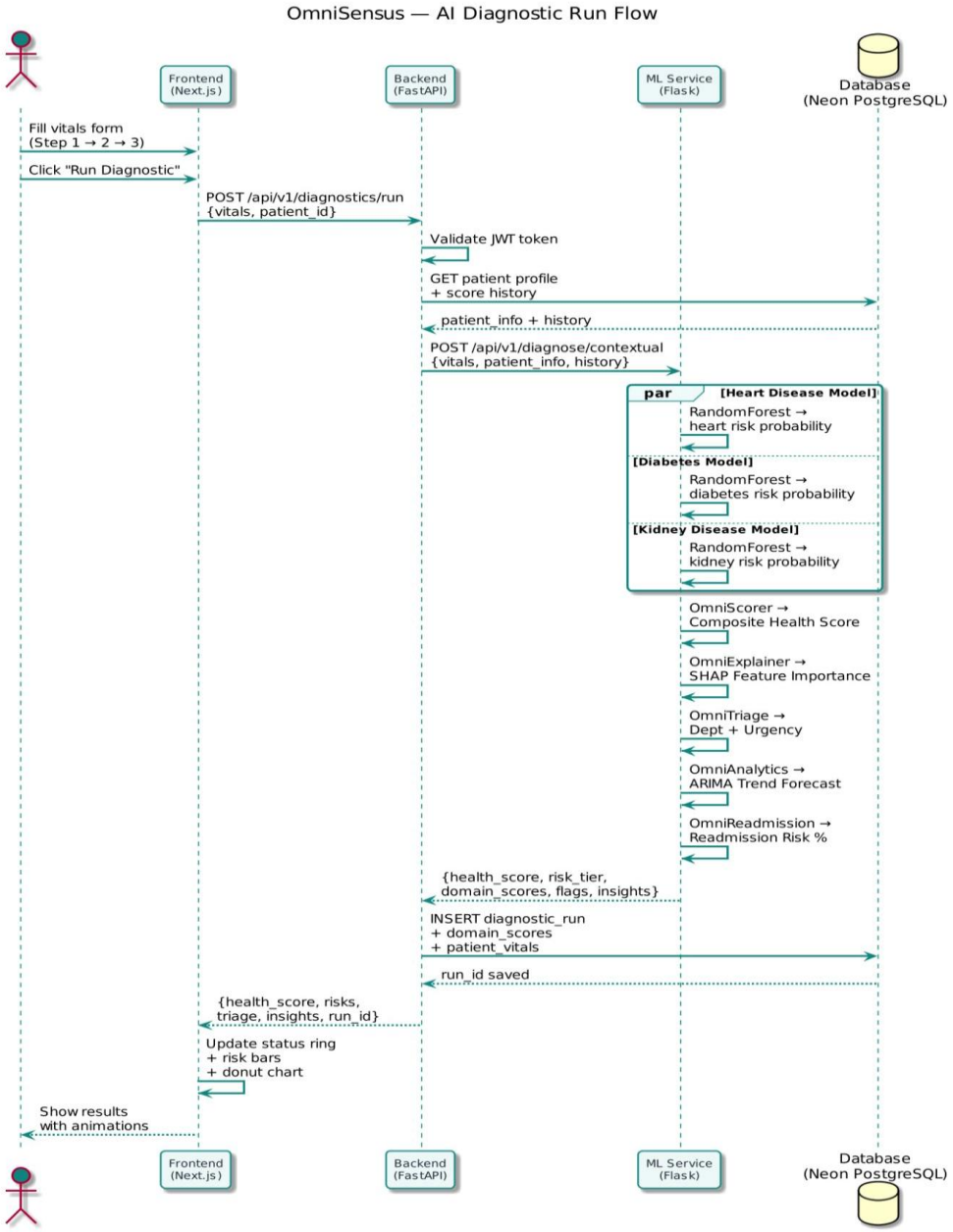


Fig 5.5 : Sequence Diagram – AI Diagnostic Run Flow

5.4.2 Login and Authentication Flow

The login and authentication sequence diagram illustrates the complete JWT authentication flow: credential submission, database lookup, bcrypt hash verification, access and refresh token generation, role-based dashboard redirection, and the token refresh sub-flow that enables seamless session continuation without user re-login.

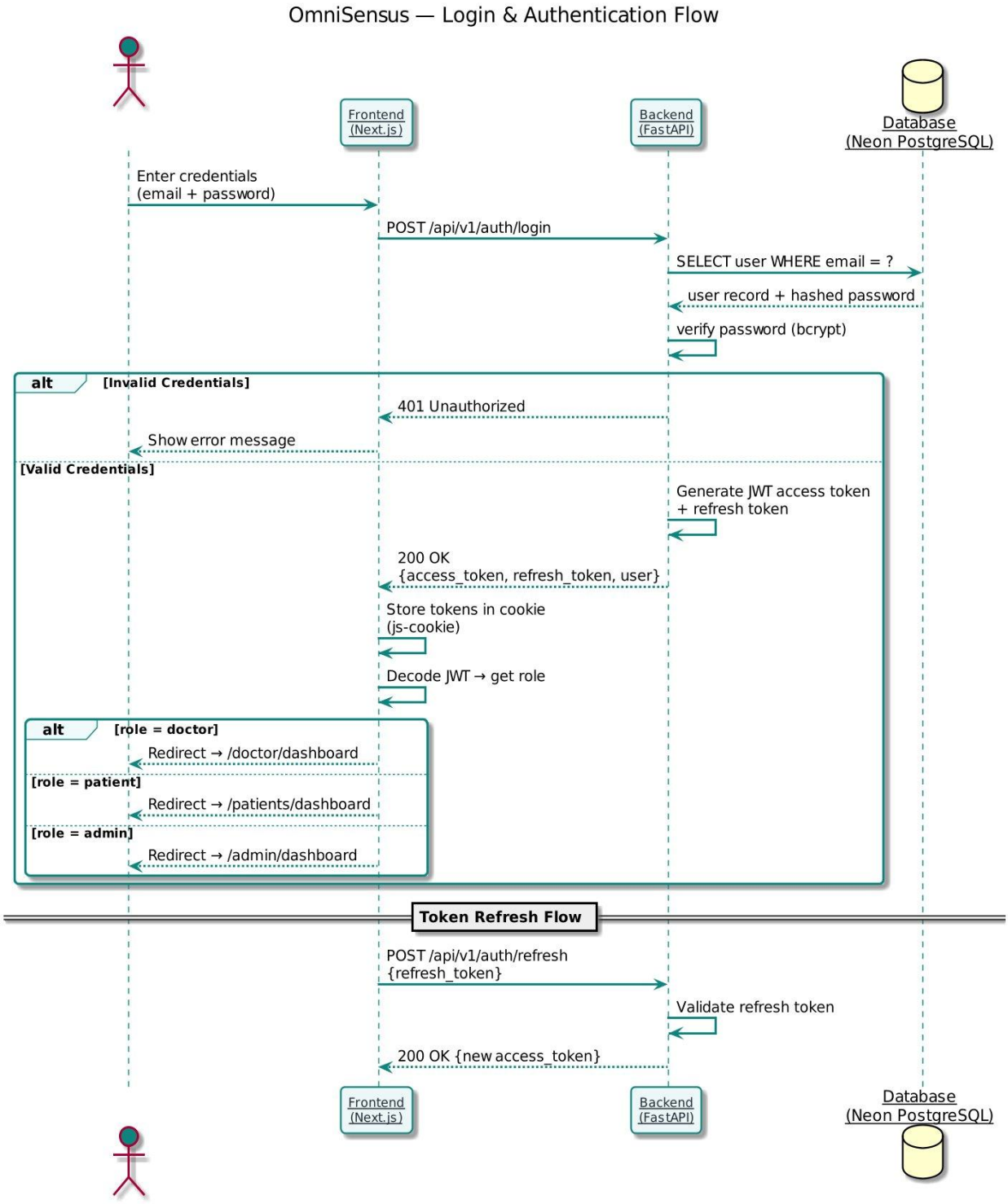


Fig 5.6 : Sequence Diagram – Login and Authentication Flow

5.5 Data Flow Diagrams

5.5.1 DFD Level 0 – Context Diagram

The Level 0 Data Flow Diagram (Context Diagram) below shows OmniSensus Medical as a single process interacting with five external entities: the Doctor (sends vitals and diagnostic requests, receives diagnostic results and AI insights), the Patient (sends login and appointment booking requests, receives health scores and medications), the Admin (sends management commands, receives analytics and audit logs), the External Keep-Alive Service (sends periodic health pings, receives 200 OK response), and the ML Service (bidirectional data exchange for inference).

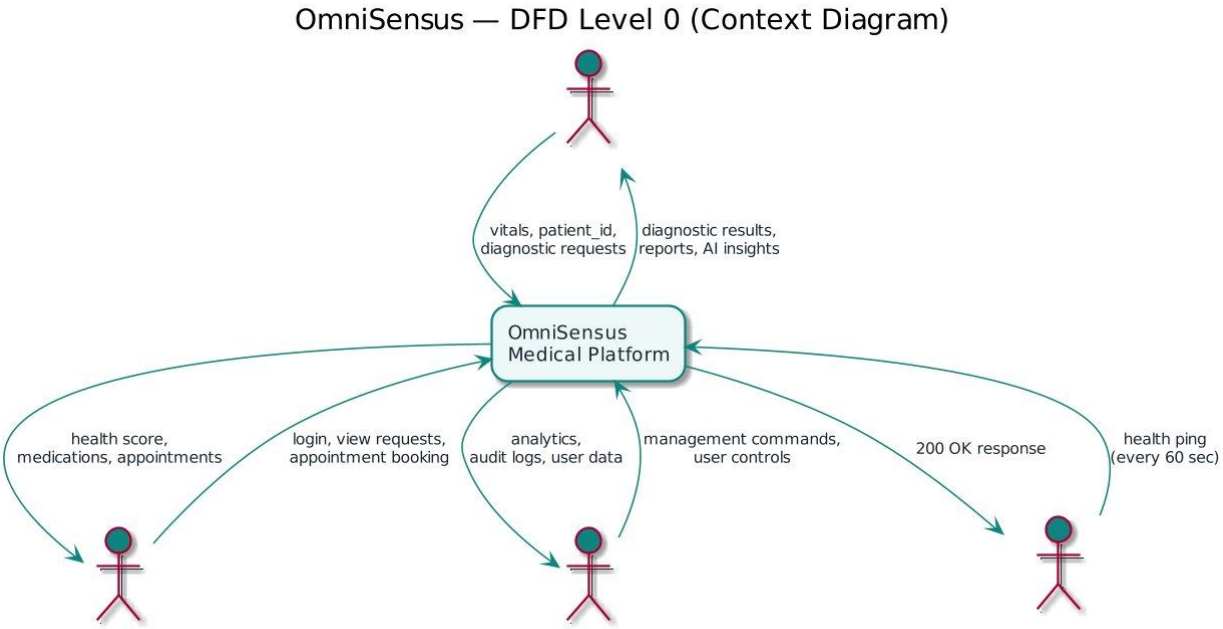


Fig 5.7 : Data Flow Diagram – Level 0 (Context Diagram)

5.5.2 DFD Level 1 – Detailed

The Level 1 DFD decomposes the OmniSensus Medical platform into seven major processes: 1.0 Authentication, 2.0 Diagnostic Engine, 3.0 Patient Data Management, 4.0 ML Risk Analysis, 5.0 Report Generation, 6.0 Notification Service, and 7.0 Admin Management. Data stores include Users DB, Notifications DB, Patients DB, Diagnostic Runs DB, and Reports DB. The diagram shows data flows between all processes and data stores, illustrating how a single diagnostic request traverses the full system.

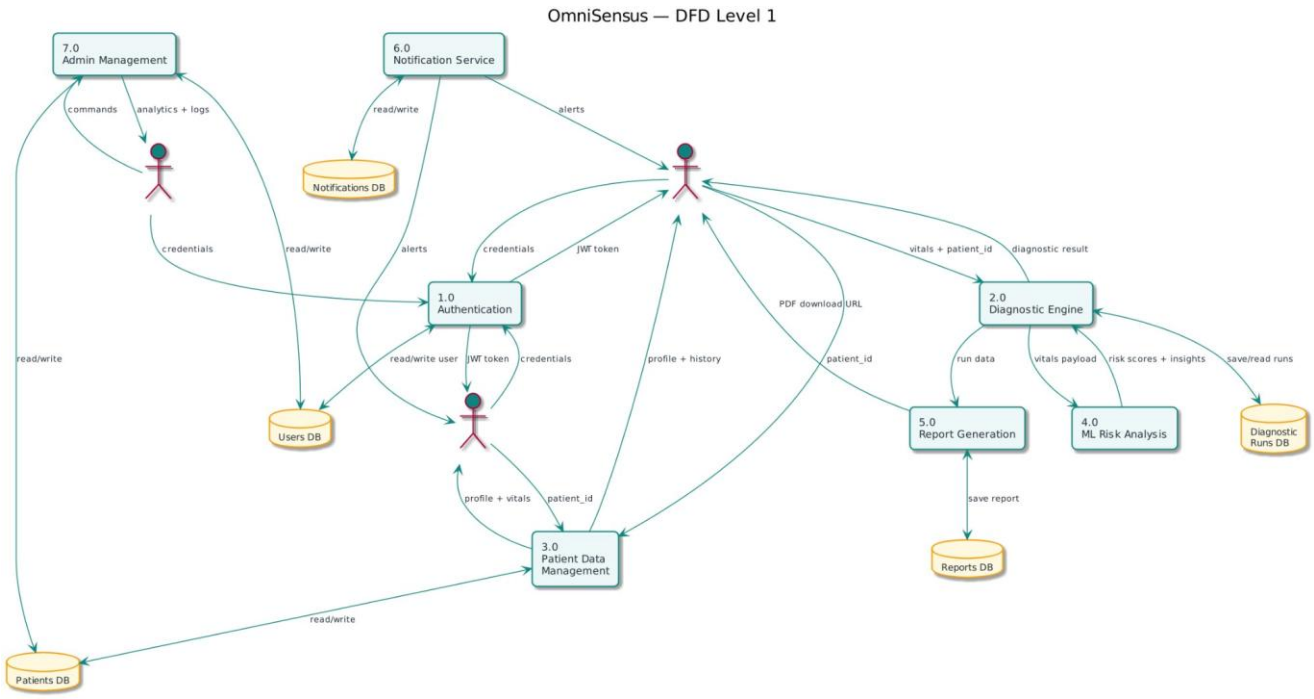


Fig 5.8 : Data Flow Diagram – Level 1 (Detailed)

5.6 Class Diagram – ML Service

The class diagram below documents the object-oriented design of the OmniSensus Medical ML microservice. It shows nine classes: OmniAnalytics (trend and forecasting), OmniTriage (department routing), OmniReadmission (readmission risk), OmniValidator (biomarker range checking), OmniPreprocessor (data pipeline), OmniMedications (drug recommendations), OmniScorer (composite health scoring), OmniTrainer (model training), OmniExplainer (SHAP attribution), ChatEngine (PharmaBot session management), and the central PatientContext class which all inference classes depend upon for adaptive weighting and comorbidity-aware thresholds.

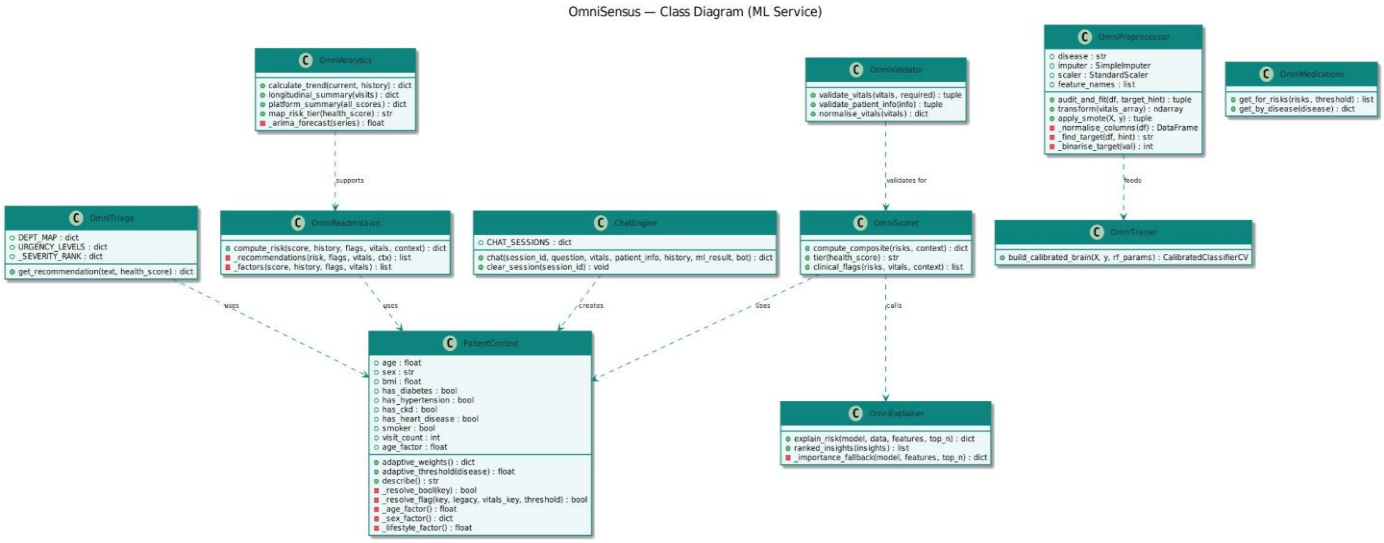


Fig 5.9 : Class Diagram – ML Service Components

5.7 ML Pipeline Diagram

The ML Pipeline Diagram below illustrates the complete machine learning workflow from raw datasets through preprocessing, training, evaluation, model export, and runtime inference. The OmniPreprocessor handles column normalisation, target detection, median imputation, standard scaling, and SMOTE class balancing. The OmniTrainer applies RandomForestClassifier with CalibratedClassifierCV (sigmoid method) to produce calibrated probability outputs. Exported model files (.pkl) are loaded at runtime by the Flask inference service, which feeds predictions through OmniScorer, OmniExplainer, OmniTriage, and OmniReadmission to produce the complete diagnostic result.

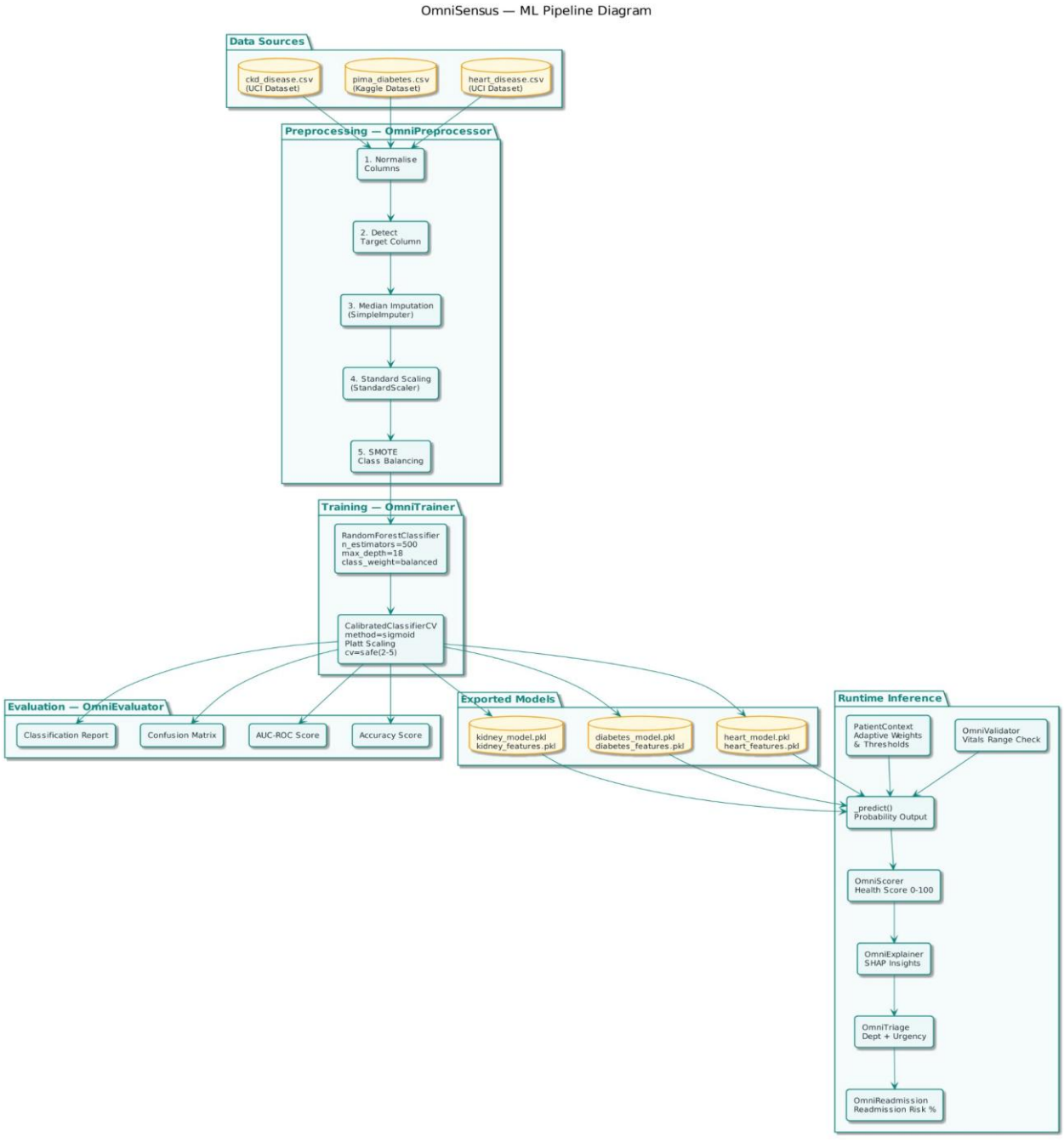


Fig 5.10 : ML Pipeline Diagram – Training and Inference

5.8 System Architecture Diagram

The system architecture diagram below provides a comprehensive view of all components of the OmniSensus Medical platform: the Next.js 14 frontend (deployed on Netlify) with its pages, features, and frontend services; the FastAPI backend (deployed on Render) with its backend middleware (JWT Auth, Role-Based Access) and eight API routers; the Neon PostgreSQL database with its core tables; and the Flask ML Service (deployed on Render) with its three ML models (Diabetes, Heart Disease, Kidney Disease), ML Engines (ARIMA Forecasting, Readmission Risk, Adaptive Thresholds), and output components (Health Score, Dept + Urgency, SHAP Explainer). External services include the Google Apps Script health monitor and Chart.js.

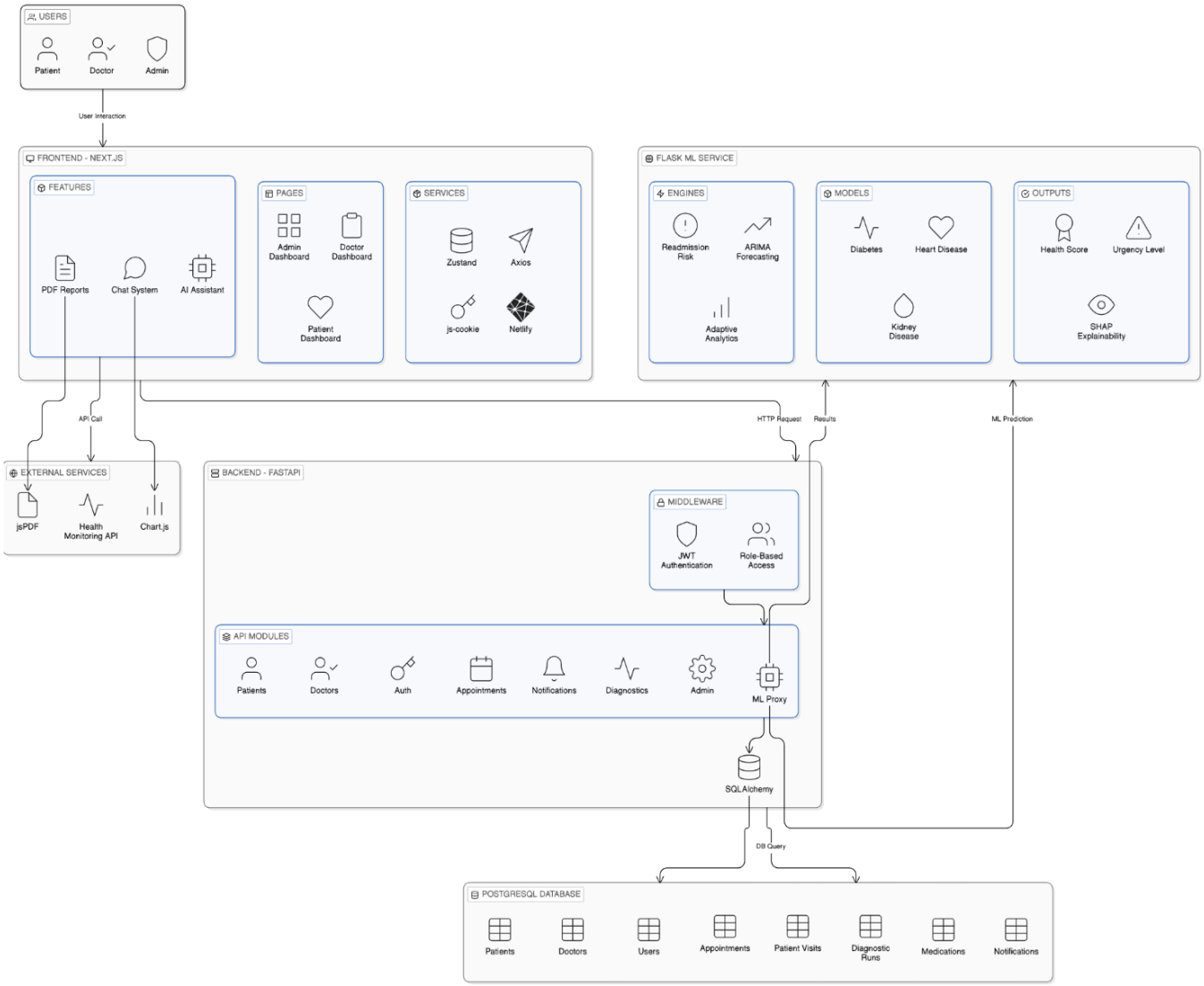


Fig 5.1 : System Architecture – Full Stack Overview

5.9 Input/Output and Interface Design

The primary data input interface is the Doctor Diagnostic Wizard, a 3-step modal form. Each step captures a specific clinical domain:

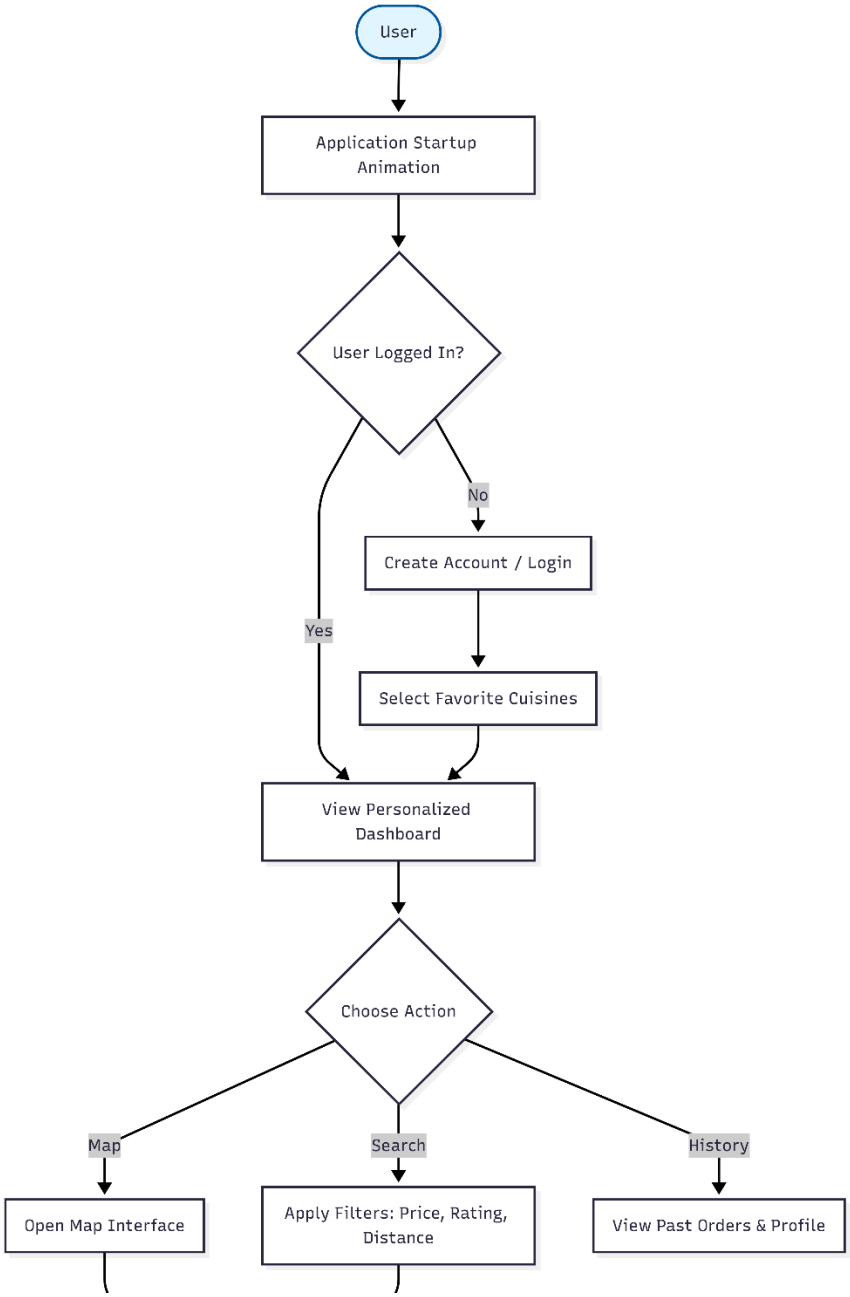
- **Step 1 – Vital Signs:** Blood pressure (systolic/diastolic), heart rate, SpO2, temperature, BMI, height, weight.
- **Step 2 – Biomarkers:** Glucose (fasting), HbA1c, insulin, post-prandial glucose, total cholesterol, LDL, HDL, triglycerides, eGFR, creatinine, BUN, urine albumin-creatinine ratio, haemoglobin, pregnancies (female patients).
- **Step 3 – Medical History:** Known diabetes, hypertension, CKD, smoker status, skin thickness, doctor notes.

Primary outputs from the system:

- **Composite Health Score** (0–100) with risk tier classification (Stable / Borderline / Critical).
- **Domain Scores:** Cardiovascular, Metabolic, and Renal scores (0–100) displayed as a radar chart.
- **Clinical Flags:** Domain-specific risk flags with severity level (info / borderline / critical).
- **SHAP Insights:** Top-5 biomarkers ranked by absolute contribution to the risk prediction.
- **Triage Urgency:** Routine / Semi-Urgent / Urgent / Critical classification with clinical note.
- **PDF Diagnostic Report:** Downloadable formatted report with all of the above.
- **Trend Charts:** Patient-specific longitudinal visualisations of key biomarkers over time.

5.10 Database Schema and Entity-Relationship Diagram

The complete database schema consists of 31 tables across two migration versions. The Entity-Relationship Diagram below illustrates the primary entities (USERS, DOCTORS, PATIENTS, DIAGNOSTIC_RUNS, PATIENT_VITALS, DOMAIN_SCORES, DIAGNOSTIC_REPORTS, MEDICATIONS, APPOINTMENTS, NOTIFICATIONS) and their relationships, cardinalities, and key foreign key constraints.



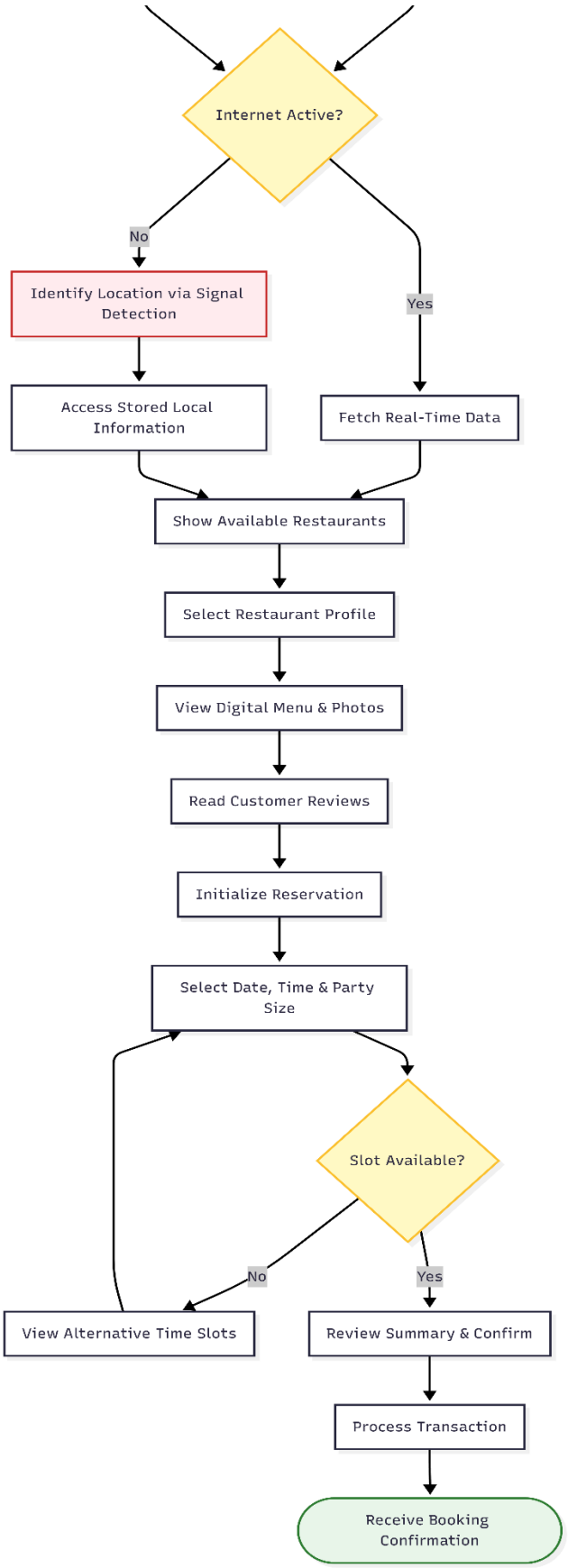


Fig 5.11 : Entity-Relationship Diagram – PostgreSQL Schema

Key tables and their roles are described below:

Table	Purpose	Key Relationships
users	Core identity for all roles	→ doctors, patients, admins
doctors	Doctor profiles and stats	← users; → patients (primary)
patients	Patient demographics, current scores	← users, → doctors
diagnostic_runs	Every AI scan with composite score	← patients, doctors → domain_scores, vitals, flags, insights (cascade)
domain_scores	CVD/Metabolic/Renal per run	← diagnostic_runs (1:1)
clinical_flags	Risk flags per domain	← diagnostic_runs
ai_insights	SHAP feature importances	← diagnostic_runs
patient_vitals	Full biomarker panel per run	← diagnostic_runs (1:1)
medications	Drug reference catalogue	→ patient_medications
patient_medications	Prescribed medications	← patients, medications, doctors
medication_adherence	Dose-level adherence logs	← patient_medications, patients
appointments	Consultations	← patients, doctors
notifications	User alerts	← users, patients (optional)
audit_logs	Immutable action log	← users (SET NULL on delete)
analytics_snapshots	Daily KPI cache	Standalone (snapshot)

Table 5.11.1: Database Tables and Their Relations

Table 1: users

Column Name	Data Type	Constraints	Description
user_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique identifier for every user across all roles
username	VARCHAR(50)	NOT NULL, UNIQUE	Login username — institutional identifier
email	VARCHAR(120)	NOT NULL, UNIQUE	User's institutional email address
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash ($2b$12) — raw password never stored
role	VARCHAR(20)	NOT NULL, CHECK IN ('admin','doctor','patient')	Role determines dashboard access and API permissions
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Soft-delete flag — FALSE suspends login
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	UTC timestamp of account creation
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	UTC timestamp of last profile update
last_login_at	TIMESTAMPTZ	NULL	UTC timestamp of most recent successful login

Table 2: admins

Column Name	Data Type	Constraints	Description
admin_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique admin record identifier
user_id	UUID	NOT NULL, FK → users(user_id) ON DELETE CASCADE	Links to master users table
full_name	VARCHAR(120)	NOT NULL	Admin's full display name
department	VARCHAR(80)	NULL	Administrative department or unit
access_level	INTEGER	NOT NULL, DEFAULT 1	Permission tier: 1=basic, 2=super-admin
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Record creation timestamp

Table 3: doctors

Column Name	Data Type	Constraints	Description
doctor_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique doctor record identifier
user_id	UUID	NOT NULL, UNIQUE, FK → users(user_id) ON DELETE CASCADE	Links to master users table
full_name	VARCHAR(120)	NOT NULL	Doctor's full display name
specialisation	VARCHAR(100)	NULL	Medical specialisation (e.g., Cardiology)
license_number	VARCHAR(60)	NULL	Medical council registration number
department	VARCHAR(80)	NULL	Hospital department affiliation
phone	VARCHAR(20)	NULL	Contact phone number
total_diagnostics	INTEGER	NOT NULL, DEFAULT 0	Running count — auto-incremented by trigger
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Record creation timestamp

Table 4: patients

Column Name	Data Type	Constraints	Description
patient_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique patient record identifier
user_id	UUID	NOT NULL, UNIQUE, FK → users(user_id) ON DELETE CASCADE	Links to master users table
full_name	VARCHAR(120)	NOT NULL	Patient's full legal name
date_of_birth	DATE	NULL	Used to compute age at time of diagnostic
age	INTEGER	NULL	Denormalised age for quick display
gender	VARCHAR(10)	NULL, CHECK IN ('M','F','Other')	Sex at birth for ML model feature input
blood_group	VARCHAR(5)	NULL	ABO/Rh blood group
phone	VARCHAR(20)	NULL	Emergency contact number
address	TEXT	NULL	Residential address
primary_doctor_id	UUID	NULL, FK → doctors(doctor_id) ON DELETE SET NULL	Assigned attending physician
current_score	NUMERIC(5,1)	NULL	Latest composite health score (0–100)
current_tier	VARCHAR(20)	NULL	Latest risk tier
last_scan_at	TIMESTAMPTZ	NULL	Timestamp of most recent diagnostic run
bmi	NUMERIC(5,2)	NULL	Cached BMI from latest vitals

known_diabetes	BOOLEAN	NOT NULL, DEFAULT FALSE	Known diabetic flag
known_hypertension	BOOLEAN	NOT NULL, DEFAULT FALSE	Known hypertension flag
known_ckd	BOOLEAN	NOT NULL, DEFAULT FALSE	Known CKD flag
known_heart_disease	BOOLEAN	NOT NULL, DEFAULT FALSE	Known CVD flag
smoker	BOOLEAN	NOT NULL, DEFAULT FALSE	Smoking status
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Record creation timestamp
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Record last-updated timestamp

Diagnostics & ML Insights

Table 5: diagnostic_runs

Column Name	Data Type	Constraints	Description
run_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique diagnostic run identifier
patient_id	UUID	NOT NULL, FK → patients(patient_id) ON DELETE CASCADE	Patient this run belongs to
doctor_id	UUID	NULL, FK → doctors(doctor_id) ON DELETE SET NULL	Doctor who initiated the diagnostic
health_score	NUMERIC(5,1)	NOT NULL	Composite health score (0–100)
risk_tier	VARCHAR(20)	NOT NULL	Stable / Borderline / Critical
run_date	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Diagnostic execution timestamp
model_version	VARCHAR(20)	NULL, DEFAULT '3.0.1'	ML model version
notes	TEXT	NULL	Physician notes
accession_number	VARCHAR(30)	UNIQUE, DEFAULT generate_accession()	OSM-ACC-XXXXXX format

Table 6: domain_scores

Column Name	Data Type	Constraints	Description
score_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique domain score record
run_id	UUID	NOT NULL, UNIQUE, FK → diagnostic_runs(run_id) CASCADE	One-to-one with diagnostic_runs
cardiovascular	NUMERIC(5,1)	NOT NULL	Cardiovascular score
metabolic	NUMERIC(5,1)	NOT NULL	Metabolic score
renal	NUMERIC(5,1)	NOT NULL	Renal score
heart_pct	NUMERIC(5,2)	NULL	Raw cardiovascular probability
diabetes_pct	NUMERIC(5,2)	NULL	Raw diabetes probability
kidney_pct	NUMERIC(5,2)	NULL	Raw kidney probability
weights_used	JSONB	NULL	Adaptive weights

Table 7: clinical_flags

Column Name	Data Type	Constraints	Description
flag_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique flag identifier
run_id	UUID	NOT NULL, FK → diagnostic_runs(run_id) CASCADE	Parent run
domain	VARCHAR(30)	NOT NULL	Domain
severity	VARCHAR(20)	NOT NULL	critical / borderline / info
message	TEXT	NOT NULL	Clinical explanation
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Timestamp

Table 8: ai_insights

Column Name	Data Type	Constraints	Description
insight_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique insight
run_id	UUID	NOT NULL, FK → diagnostic_runs(run_id) CASCADE	Parent run
domain	VARCHAR(30)	NOT NULL	Model domain
rank_position	INTEGER	NOT NULL, CHECK (1 ≤ rank ≤ 10)	Importance rank
feature_name	VARCHAR(80)	NOT NULL	Biomarker name
shap_value	NUMERIC(10,4)	NOT NULL	SHAP value
feature_value	NUMERIC(10,4)	NULL	Actual value
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Timestamp

Clinical Data & Vitals

Table 9: patient_vitals

Column Name	Data Type	Constraints	Description
vitals_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique vitals record
run_id	UUID	NOT NULL, UNIQUE, FK → diagnostic_runs CASCADE	One-to-one
patient_id	UUID	NOT NULL, FK → patients CASCADE	Patient
glucose	NUMERIC(6,1)	NULL	Blood glucose
hba1c	NUMERIC(4,1)	NULL	HbA1c
insulin	NUMERIC(7,2)	NULL	Insulin
blood_pressure_sys	INTEGER	NULL	Systolic BP
blood_pressure_dia	INTEGER	NULL	Diastolic BP
heart_rate	INTEGER	NULL	Heart rate
cholesterol_total	NUMERIC(6,1)	NULL	Total cholesterol
ldl	NUMERIC(6,1)	NULL	LDL
hdl	NUMERIC(6,1)	NULL	HDL
triglycerides	NUMERIC(6,1)	NULL	Triglycerides
egfr	NUMERIC(6,1)	NULL	eGFR
creatinine	NUMERIC(5,2)	NULL	Creatinine
bun	NUMERIC(5,1)	NULL	BUN
urine_albumin	NUMERIC(7,1)	NULL	UACR (Urine Albumin Creatinine)
bmi	NUMERIC(5,2)	NULL	BMI
height_cm	NUMERIC(5,1)	NULL	Height
weight_kg	NUMERIC(5,1)	NULL	Weight
spo2	NUMERIC(4,1)	NULL	Oxygen saturation

temperature	NUMERIC(4,1)	NULL	Temperature
hemoglobin	NUMERIC(4,1)	NULL	Hemoglobin
pregnancies	INTEGER	NULL	Pregnancies
skin_thickness	NUMERIC(4,1)	NULL	Skin thickness
crp	NUMERIC(6,2)	NULL	CRP
recorded_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Timestamp

Medication & Treatment

Table 10: medications

Column Name	Data Type	Constraints	Description
medication_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique drug
name	VARCHAR(120)	NOT NULL, UNIQUE	Drug name
drug_class	VARCHAR(80)	NOT NULL	Class
dosage	VARCHAR(80)	NULL	Dosage
indication	TEXT	NULL	Indication
contraindications	TEXT	NULL	Contraindications
side_effects	TEXT	NULL	Side effects
guideline_source	VARCHAR(100)	NULL	Source
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Timestamp

Table 11: patient_medications

Column Name	Data Type	Constraints	Description
patient_med_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Prescription ID
patient_id	UUID	NOT NULL, FK → patients CASCADE	Patient
medication_id	UUID	NOT NULL, FK → medications RESTRICT	Drug
prescribed_by	UUID	NULL, FK → doctors SET NULL	Doctor
dose_actual	VARCHAR(60)	NULL	Dose
frequency	VARCHAR(40)	NULL	Frequency
scheduled_time	TIME	NULL	Time
start_date	DATE	NOT NULL	Start
end_date	DATE	NULL	End
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Active
notes	TEXT	NULL	Notes
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Timestamp

Operations & Logistics

Table 12: appointments

Column Name	Data Type	Constraints	Description
appointment_id	UUID	PRIMARY KEY	Unique appointment
patient_id	UUID	FK → patients CASCADE	Patient
doctor_id	UUID	FK → doctors CASCADE	Doctor
scheduled_at	TIMESTAMPTZ	NOT NULL	Date/time
type	VARCHAR(40)	DEFAULT 'Consultation'	Type
status	VARCHAR(20)	DEFAULT 'pending'	Status
location	VARCHAR(120)	NULL	Location
notes	TEXT	NULL	Notes
created_at	TIMESTAMPTZ	DEFAULT NOW()	Created
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Updated

Table 13: diagnostic_reports

Column Name	Data Type	Constraints	Description
report_id	UUID	PRIMARY KEY	Unique report
run_id	UUID	FK → diagnostic_runs CASCADE	Run
patient_id	UUID	FK → patients CASCADE	Patient
report_filename	VARCHAR(200)	NOT NULL	File
pdf_url	TEXT	NULL	URL
generated_at	TIMESTAMPTZ	DEFAULT NOW()	Generated
generated_by	UUID	FK → doctors SET NULL	Doctor

Table 14: visit_history

Column Name	Data Type	Constraints	Description
visit_id	UUID	PRIMARY KEY	Visit
patient_id	UUID	FK → patients CASCADE	Patient
run_id	UUID	FK → diagnostic_runs SET NULL	Run
visit_date	DATE	NOT NULL	Date
visit_type	VARCHAR(60)	NULL	Type
health_score	NUMERIC(5,1)	NULL	Score
risk_tier	VARCHAR(20)	NULL	Tier
doctor_id	UUID	FK → doctors SET NULL	Doctor
notes	TEXT	NULL	Notes
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Infrastructure & Support

Table 15: notifications

Column Name	Data Type	Constraints	Description
notification_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique notification
user_id	UUID	NOT NULL, FK → users(user_id) CASCADE	Recipient user
patient_id	UUID	NULL, FK → patients CASCADE	Related patient
type	VARCHAR(40)	NOT NULL	Type
title	VARCHAR(150)	NOT NULL	Headline
message	TEXT	NOT NULL	Body
severity	VARCHAR(20)	NULL	Severity
is_read	BOOLEAN	NOT NULL, DEFAULT FALSE	Read flag
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Timestamp

Table 16: audit_logs

Column Name	Data Type	Constraints	Description
log_id	UUID	PRIMARY KEY	Log ID
user_id	UUID	FK → users SET NULL	Actor
action	VARCHAR(80)	NOT NULL	Action
entity_type	VARCHAR(50)	NULL	Entity type
entity_id	UUID	NULL	Entity ID
ip_address	INET	NULL	IP
details	JSONB	NULL	Event details
tx_id	VARCHAR(20)	UNIQUE, DEFAULT generate_tx_id()	Transaction ID
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Timestamp

Table 17: beds

Column Name	Data Type	Constraints	Description
bed_id	UUID	PRIMARY KEY	Bed
bed_number	VARCHAR(10)	NOT NULL, UNIQUE	Bed number
ward	VARCHAR(60)	NOT NULL	Ward
status	VARCHAR(20)	DEFAULT 'available'	Status
patient_id	UUID	FK → patients SET NULL	Occupant
notes	TEXT	NULL	Notes
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Table 18: equipment

Column Name	Data Type	Constraints	Description
equipment_id	UUID	PRIMARY KEY	Equipment
name	VARCHAR(100)	NOT NULL	Name
category	VARCHAR(60)	NOT NULL	Category
status	VARCHAR(20)	DEFAULT 'operational'	Status
location	VARCHAR(80)	NULL	Location
utilisation_pct	INTEGER	DEFAULT 0	Usage
next_maintenance	DATE	NULL	Maintenance
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Table 19: staff_oncall

Column Name	Data Type	Constraints	Description
oncall_id	UUID	PRIMARY KEY	ID
doctor_id	UUID	FK → doctors CASCADE	Doctor
full_name	VARCHAR(120)	NOT NULL	Name
specialisation	VARCHAR(100)	NULL	Specialisation
department	VARCHAR(80)	NULL	Department
shift_date	DATE	NOT NULL	Date
shift_start	TIME	NOT NULL	Start
shift_end	TIME	NOT NULL	End
shift_label	VARCHAR(20)	NULL	Label
patient_count	INTEGER	DEFAULT 0	Load
is_available	BOOLEAN	DEFAULT TRUE	Availability

Technical Monitoring

Table 20: model_events

Column Name	Data Type	Constraints	Description
event_id	UUID	PRIMARY KEY	Event
run_id	UUID	FK → diagnostic_runs SET NULL	Run
model_name	VARCHAR(40)	NOT NULL	Model
event_type	VARCHAR(30)	NOT NULL	Type
status	VARCHAR(20)	DEFAULT 'success'	Status
latency_ms	INTEGER	NULL	Latency
model_version	VARCHAR(20)	NULL	Version
error_message	TEXT	NULL	Error
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Table 21: support_engineers

Column Name	Data Type	Constraints	Description
engineer_id	UUID	PRIMARY KEY	Engineer
user_id	UUID	FK → users CASCADE	User
full_name	VARCHAR(120)	NOT NULL	Name
expertise	VARCHAR(80)	NULL	Expertise
is_active	BOOLEAN	DEFAULT TRUE	Status
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Table 22: dev_reports

Column Name	Data Type	Constraints	Description
dev_report_id	UUID	PRIMARY KEY	Report
reported_by	UUID	FK → users SET NULL	Reporter
title	VARCHAR(200)	NOT NULL	Title
description	TEXT	NULL	Description
severity	VARCHAR(20)	NULL	Severity
status	VARCHAR(20)	DEFAULT 'open'	Status
created_at	TIMESTAMPTZ	DEFAULT NOW()	Created
resolved_at	TIMESTAMPTZ	NULL	Resolved

Version 2: Patient Engagement & Analytics

Table 23: user_preferences

Column Name	Data Type	Constraints	Description
pref_id	UUID	PRIMARY KEY	Preference
user_id	UUID	UNIQUE, FK → users CASCADE	User
theme	VARCHAR(20)	DEFAULT 'light'	Theme
language	VARCHAR(10)	DEFAULT 'en'	Language
email_notifications	BOOLEAN	DEFAULT TRUE	Email
push_alerts	BOOLEAN	DEFAULT TRUE	Push
dashboard_layout	JSONB	NULL	Layout
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Table 24: patient_goals

Column Name	Data Type	Constraints	Description
goal_id	UUID	PRIMARY KEY	Goal
patient_id	UUID	FK → patients CASCADE	Patient
goal_type	VARCHAR(60)	NOT NULL	Type
target_value	NUMERIC(8,2)	NULL	Target
unit	VARCHAR(20)	NULL	Unit
start_date	DATE	NOT NULL	Start
target_date	DATE	NULL	Target date
status	VARCHAR(20)	DEFAULT 'active'	Status
notes	TEXT	NULL	Notes
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Table 25: health_tips

Column Name	Data Type	Constraints	Description
tip_id	UUID	PRIMARY KEY	Tip
category	VARCHAR(40)	NOT NULL	Category
title	VARCHAR(150)	NOT NULL	Title
content	TEXT	NOT NULL	Content
risk_tier	VARCHAR(20)	NULL	Tier
source	VARCHAR(100)	NULL	Source
is_active	BOOLEAN	DEFAULT TRUE	Active
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Table 26: medication_adherence

Column Name	Data Type	Constraints	Description
adherence_id	UUID	PRIMARY KEY	ID
patient_med_id	UUID	FK → patient_meds CASCADE	Prescription
patient_id	UUID	FK → patients CASCADE	Patient
scheduled_date	DATE	NOT NULL	Date
taken_at	TIMESTAMPTZ	NULL	Taken time
status	VARCHAR(20)	DEFAULT 'pending'	Status
notes	TEXT	NULL	Notes
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Table 27: bed_history

Column Name	Data Type	Constraints	Description
history_id	UUID	PRIMARY KEY	History
bed_id	UUID	FK → beds CASCADE	Bed
patient_id	UUID	FK → patients SET NULL	Patient
status	VARCHAR(20)	NOT NULL	Status
changed_at	TIMESTAMPTZ	DEFAULT NOW()	Time
changed_by	UUID	FK → users SET NULL	User
notes	TEXT	NULL	Notes

Table 28: resource_alerts

Column Name	Data Type	Constraints	Description
alert_id	UUID	PRIMARY KEY	Alert
resource_type	VARCHAR(40)	NOT NULL	Type
resource_id	UUID	NULL	Resource
severity	VARCHAR(20)	NOT NULL	Severity
message	TEXT	NOT NULL	Message
is_resolved	BOOLEAN	DEFAULT FALSE	Status
created_at	TIMESTAMPTZ	DEFAULT NOW()	Created
resolved_at	TIMESTAMPTZ	NULL	Resolved

Table 29: analytics_snapshots

Column Name	Data Type	Constraints	Description
snapshot_id	UUID	PRIMARY KEY	Snapshot
snapshot_date	DATE	UNIQUE	Date
total_patients	INTEGER	DEFAULT 0	Total
critical_count	INTEGER	DEFAULT 0	Critical
borderline_count	INTEGER	DEFAULT 0	Borderline
stable_count	INTEGER	DEFAULT 0	Stable
mean_score	NUMERIC(5,1)	NULL	Mean
scans_today	INTEGER	DEFAULT 0	Scans
new_patients	INTEGER	DEFAULT 0	New
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Communication & Chat

Table 30: chat_sessions

Column Name	Data Type	Constraints	Description
session_id	VARCHAR(80)	PRIMARY KEY	Session
user_id	UUID	FK → users SET NULL	User
context	VARCHAR(20)	DEFAULT 'general'	Context
created_at	TIMESTAMPTZ	DEFAULT NOW()	Created
last_active	TIMESTAMPTZ	DEFAULT NOW()	Active
message_count	INTEGER	DEFAULT 0	Count

Table 31: chat_messages

Column Name	Data Type	Constraints	Description
message_id	UUID	PRIMARY KEY	Message
session_id	VARCHAR(80)	FK → chat_sessions CASCADE	Session
role	VARCHAR(10)	CHECK ('user','assistant')	Role
content	TEXT	NOT NULL	Content
intent	VARCHAR(40)	NULL	Intent
mode	VARCHAR(20)	NULL	Mode
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

Automated Triggers (12):

Trigger Name	Table	Purpose
trg_sync_patient_score	diagnostic_runs	Updates patient score/tier after every diagnostic run
trg_doctor_diagnostics	diagnostic_runs	Increments total_diagnostics count for the doctor
trg_update_adherence_pct	medication_adherence	Recalculates medication adherence percentage
trg_set_accession_number	diagnostic_runs	Generates OSM-ACC-XXXXXX accession number
trg_set_audit_tx_id	audit_logs	Generates unique transaction ID
audit_no_update	audit_logs	Prevents UPDATE on audit records (immutability)
audit_no_delete	audit_logs	Prevents DELETE on audit records (immutability)
trg_patients_updated_at	patients	Auto-updates timestamp on patient record change
trg_users_updated_at	users	Auto-updates timestamp on user record change
trg_appointments_updated_at	appointments	Auto-updates timestamp on appointment change
trg_equipment_updated_at	equipment	Auto-updates timestamp on equipment change
trg_beds_updated_at	beds	Auto-updates timestamp on bed record change

5.11 ML Model Architecture

Three binary classification models are trained and deployed:

Domain	Algorithm	Input Features	Dataset Size	AUC Score
Cardiovascular	CalibratedClassifierCV (RandomForest, n=500, max_depth=18)	Age, BP, cholesterol, LDL, CRP, SpO2, BMI, smoking, family history	70,000 rows	91.3%
Metabolic (Diabetes)	CalibratedClassifierCV (RandomForest, n=500, max_depth=18)	Pregnancies, glucose, BP, skin thickness, insulin, BMI, pedigree, age	768 rows (PIMA)	82.1%
Renal (CKD)	CalibratedClassifierCV (RandomForest, n=500, max_depth=18)	eGFR, creatinine, BUN, haemoglobin, sodium, potassium, albumin, BP, RBC, WBC, anaemia, oedema	400 rows	100.0%

Table 5.12.1: ML Model Performance Summary

Composite Health Score Formula:

risk = (heart_pct × 0.40) + (diabetes_pct × 0.35) + (kidney_pct × 0.25)

health_score = max(5, min(100, round((1 − composite_risk) × 100)))

Risk tiers: Stable (health_score 66–100), Borderline (health_score 41–65), Critical (health_score 0–40). An acute SpO2 override caps the score at 38 (Critical) if SpO2 < 90%, regardless of ML outputs.

CHAPTER 6. IMPLEMENTATION

6.1 Module Specifications

6.1.1 User Role and Permission Matrix

Feature / Action	Admin	Doctor	Patient
View all patients	Yes	Own patients only	Own profile only
Run AI diagnostic	No	Yes	No
View AI results	Yes (all)	Yes (own patients)	Yes (own)
Manage medications	No	Yes (prescribe)	View only
Book appointments	No	Yes	Yes
View audit logs	Yes	No	No
Access model accuracy	Yes	No	No
Manage users	Yes	No	No
View bed occupancy	Yes	No	No
Access PharmaBot	No	Yes	Yes
Download PDF reports	Yes	Yes	Yes (own)
Edit preferences	Yes	Yes	Yes

Table 6.1.1: User Role and Permission Matrix

6.1.2 Authentication Module

16. Client sends POST /auth/login with username and password.
17. Backend queries users table, verifies bcrypt hash ($2b$12 cost factor).
18. On success, generates access token (30-minute expiry) and refresh token (7-day expiry) signed with JWT_SECRET.
19. Frontend stores tokens and attaches Bearer token to every API request via Axios interceptor.
20. On 401 response, interceptor automatically calls POST /auth/refresh to obtain new access token without user re-login.
21. Role is embedded in JWT payload and decoded server-side by the authentication dependency injected into every protected endpoint.

6.1.3 Diagnostic Module

22. Doctor completes the 3-step wizard form in the frontend.
23. Frontend sends POST /diagnostics to FastAPI with the complete biomarker payload and patient_id.
24. FastAPI validates input (Pydantic model), retrieves patient context from PostgreSQL, and forwards to Flask ML service.
25. Flask ML service: validates and normalises inputs → runs three classifiers → computes composite score → generates SHAP rankings → computes triage urgency → returns structured result.
26. FastAPI receives ML result and writes diagnostic_runs, patient_vitals, domain_scores, clinical_flags, and ai_insights to PostgreSQL in a single transaction.
27. Database trigger trg_sync_patient_score automatically updates patients.current_score, current_tier, and last_scan_at.
28. FastAPI creates a notification for the doctor (and patient if Critical tier) and returns the complete result to the frontend.
29. Frontend renders the health score ring, domain radar chart, SHAP bar chart, and clinical flags list.

6.1.4 Seed Data Module

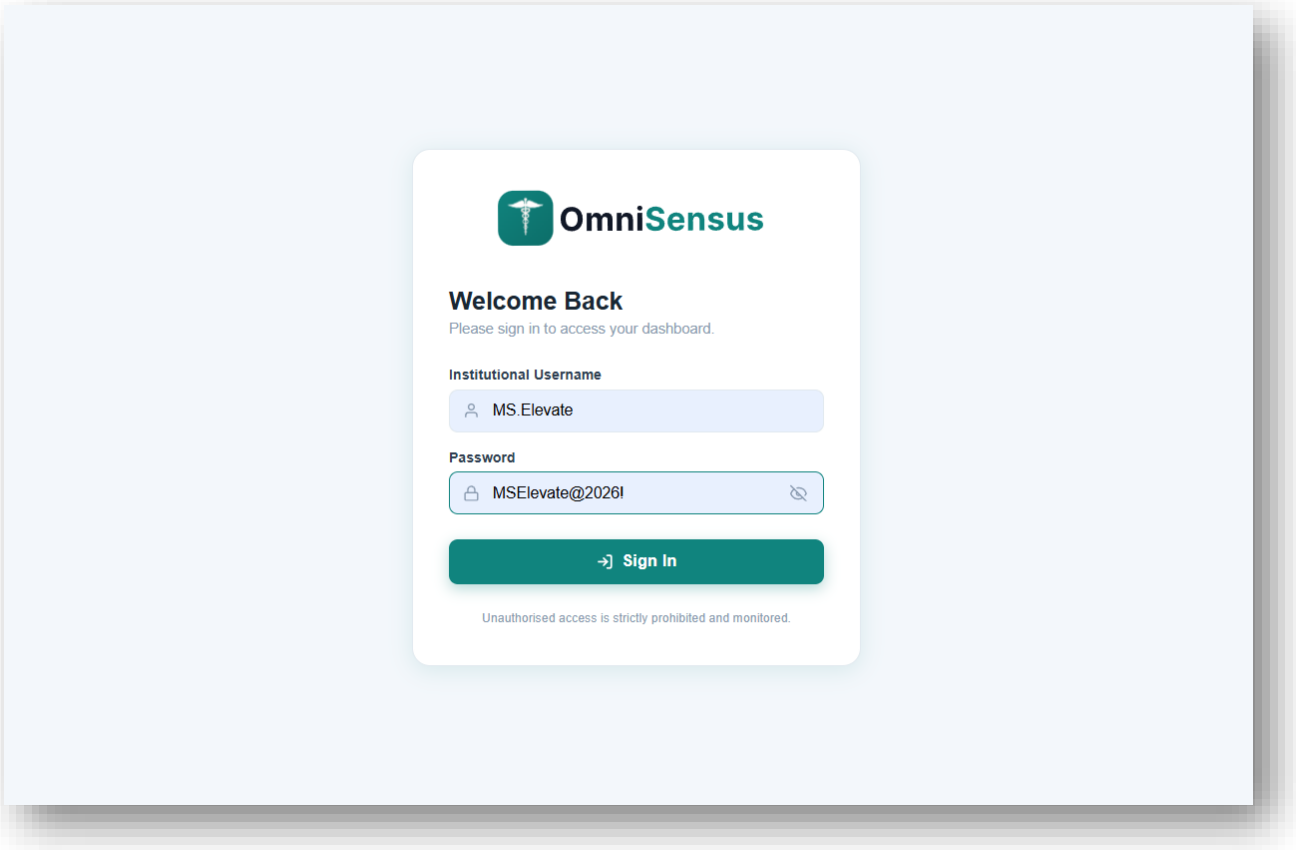
Table	Count	Notes
users	22	1 admin, 5 doctors, 16 patients
diagnostic_runs	67	2–8 runs per patient over 8 months
patient_vitals	67	Full biomarker panel per run
domain_scores	67	CVD, Metabolic, Renal per run
clinical_flags	~150	Risk flags per domain
ai_insights	~335	Top-5 SHAP features per run
patient_medications	~60	Active prescriptions
appointments	~48	Past and upcoming
notifications	~64	Per user alerts
analytics_snapshots	15	Daily platform KPI cache

Table 6.2.1: Seed Data Summary

6.2 RESULT ANALYSIS AND OUTCOMES

6.2.1 Authentication and Login

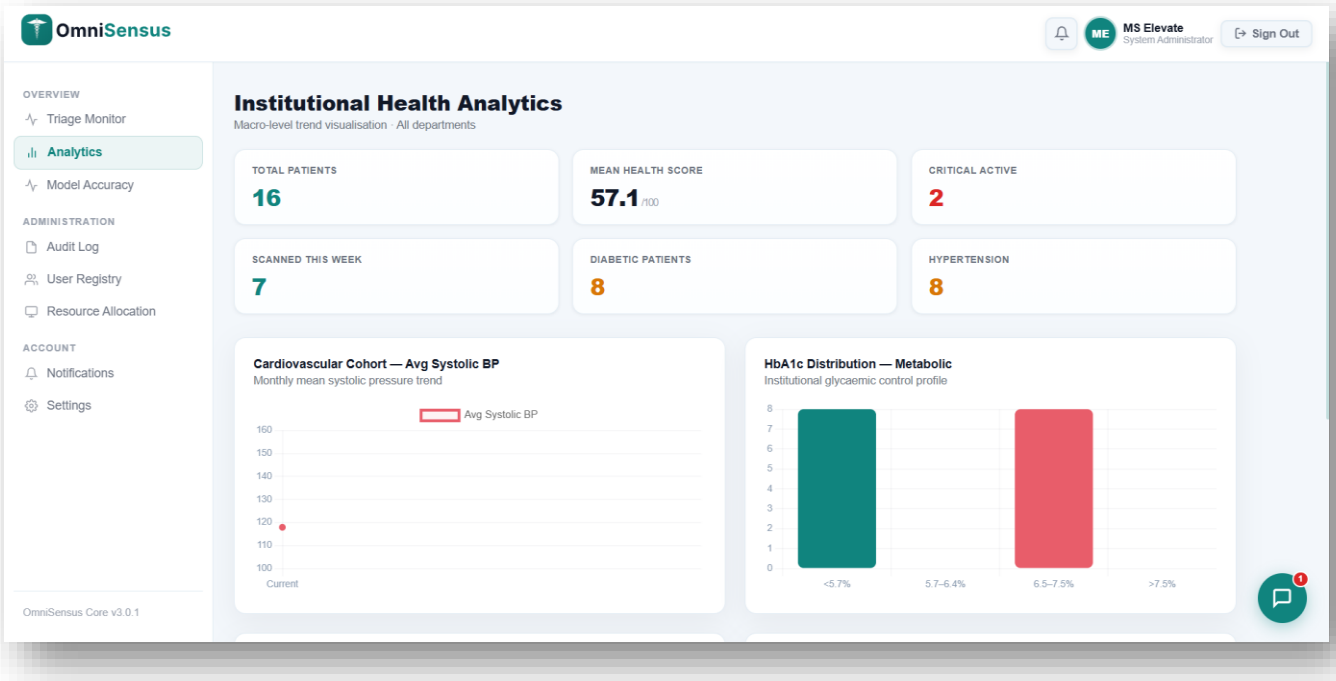
The authentication page provides separate login tabs for Admin, Doctor, and Patient roles. Each role is distinguished by its JWT payload, which controls all downstream data access and UI rendering. On successful login, the user is redirected to their role-specific dashboard.



Authentication and Login

6.2.2 Admin Dashboard – Platform Analytics

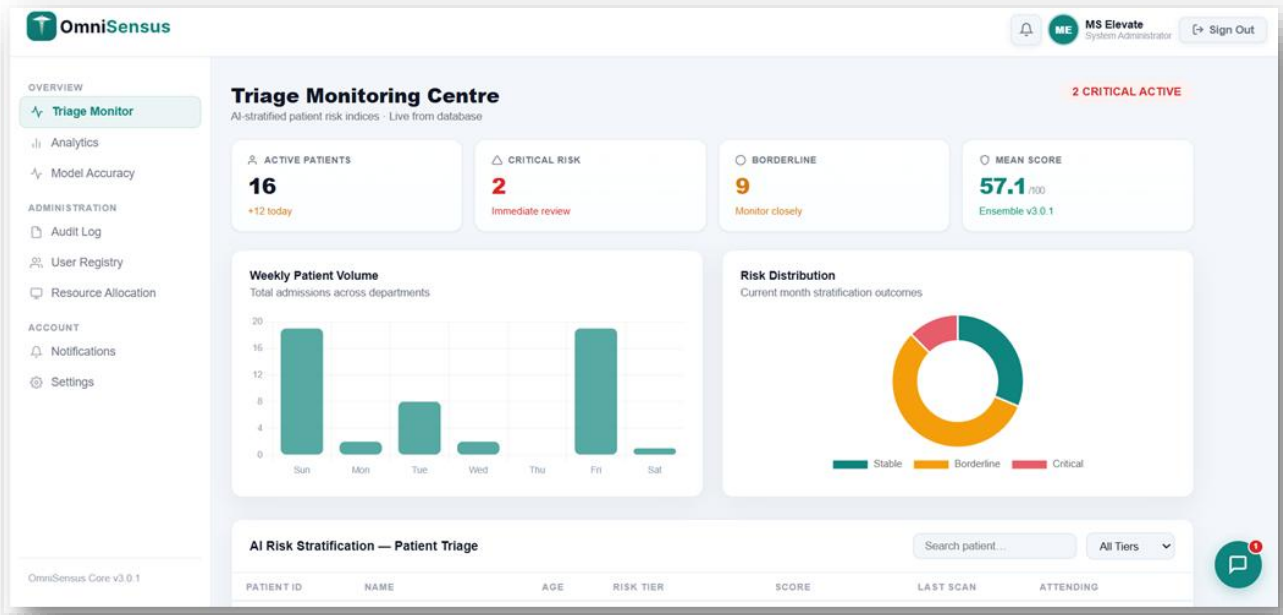
The Admin dashboard displays real-time platform KPIs including total patient count, critical patient count, mean health score, and number of scans completed this week. These figures are served from the v_platform_analytics PostgreSQL view. A line chart shows the 15-day health score trend from the analytics_snapshots table.

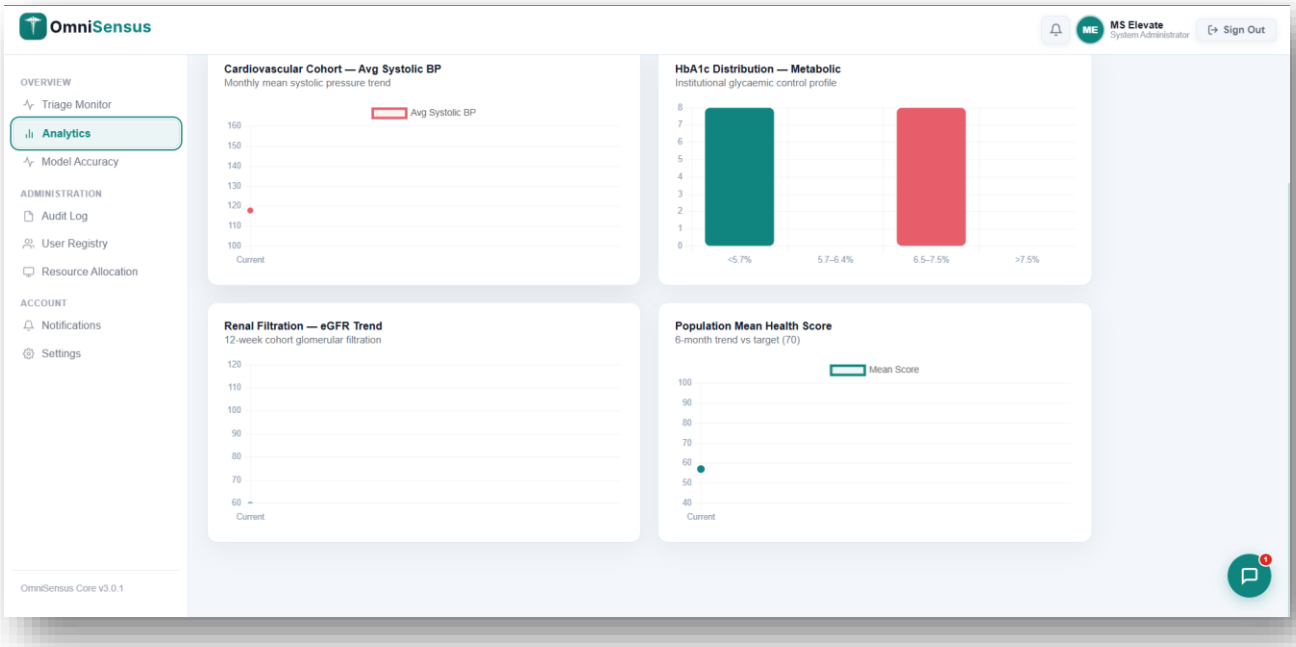


Admin Dashboard – Platform Analytics

6.2.3 Admin Triage Monitor

The triage monitor displays all patients sorted by risk tier (Critical first) and health score (ascending within tier). Each row shows the patient name, current score ring, tier badge, last scan timestamp, domain scores (CVD/Metabolic/Renal), and assigned doctor. Admins can filter by tier and search by patient name using v_triage_queue.

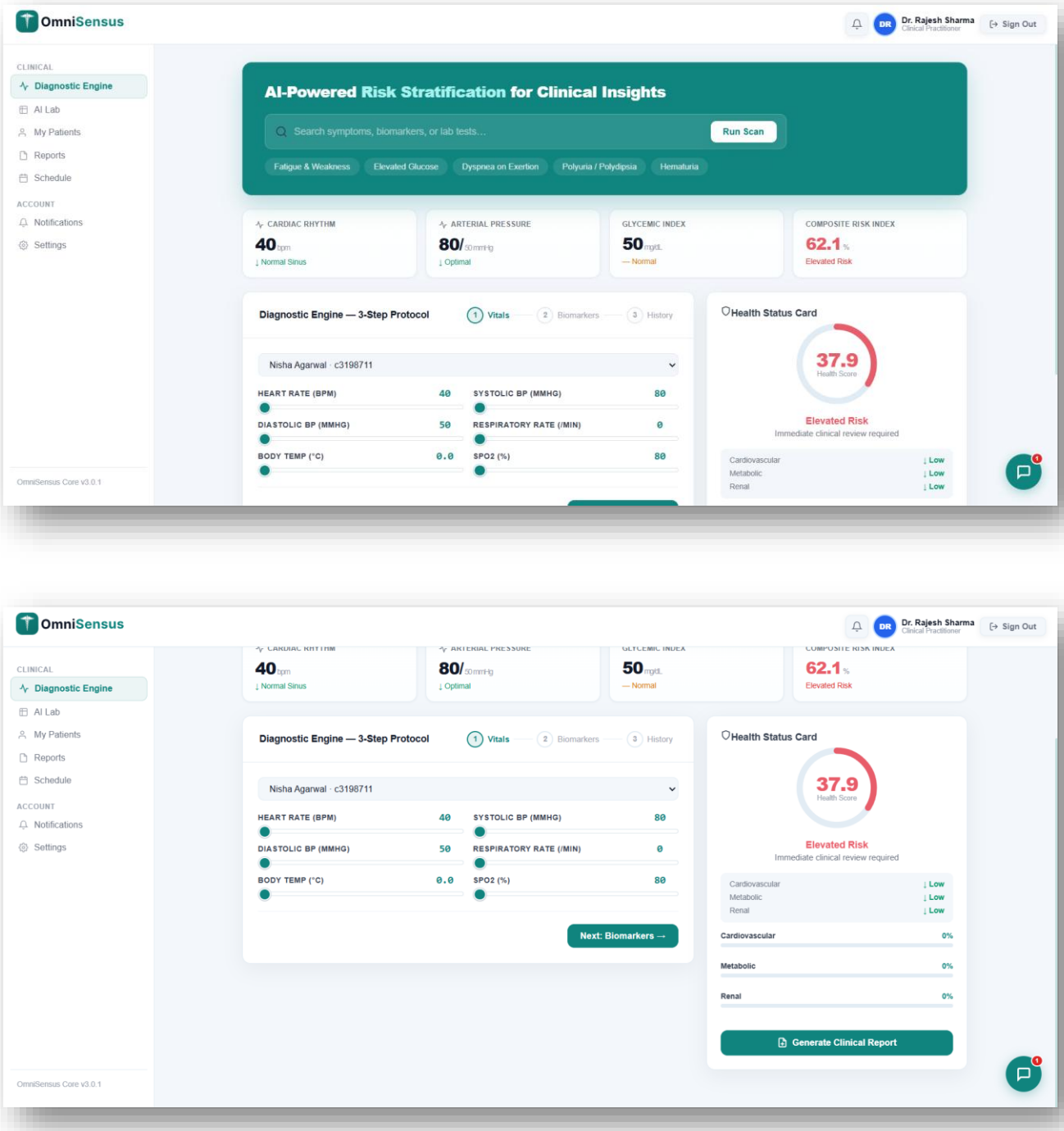




Admin Triage Monitor

6.2.4 Doctor Diagnostic Wizard

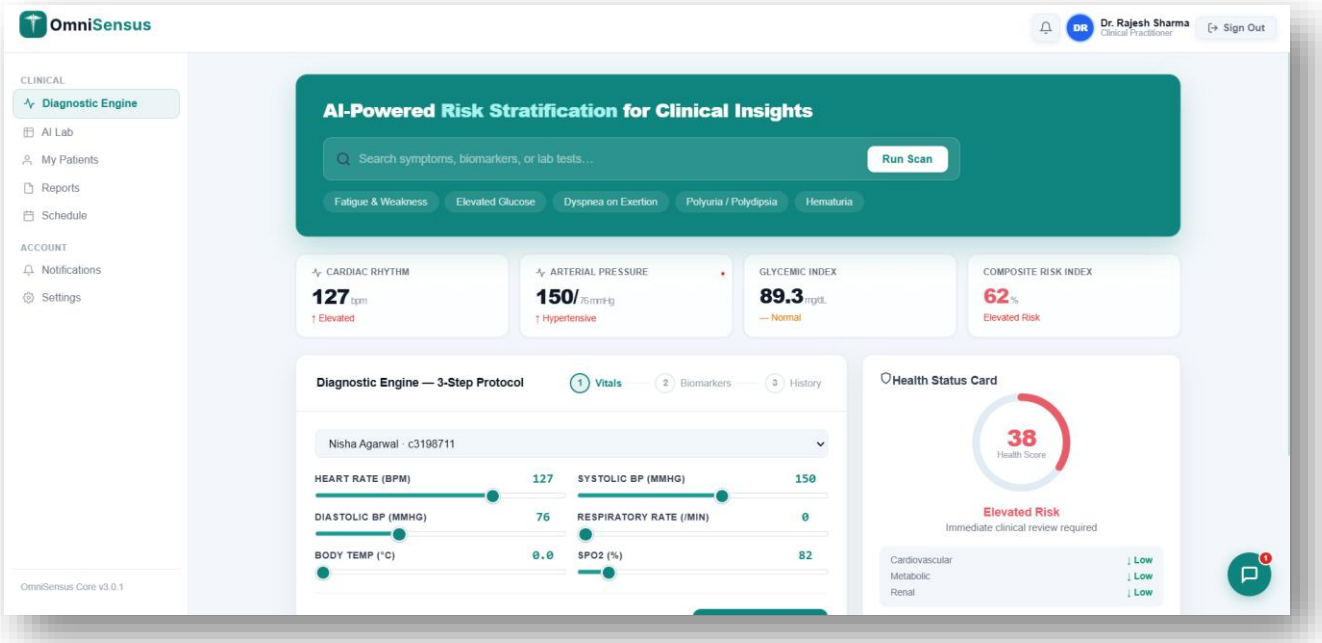
The 3-step diagnostic wizard guides doctors through Vital Signs, Biomarkers, and Medical History. Each field displays a reference range tooltip. On Step 3 completion, the form submits to POST /diagnostics. The API call takes approximately 300–420 ms (ML inference latency). The result renders immediately in the Results modal — health score ring animates from 0 to the computed value, domain bars fill progressively, and SHAP features are ranked in descending importance order.

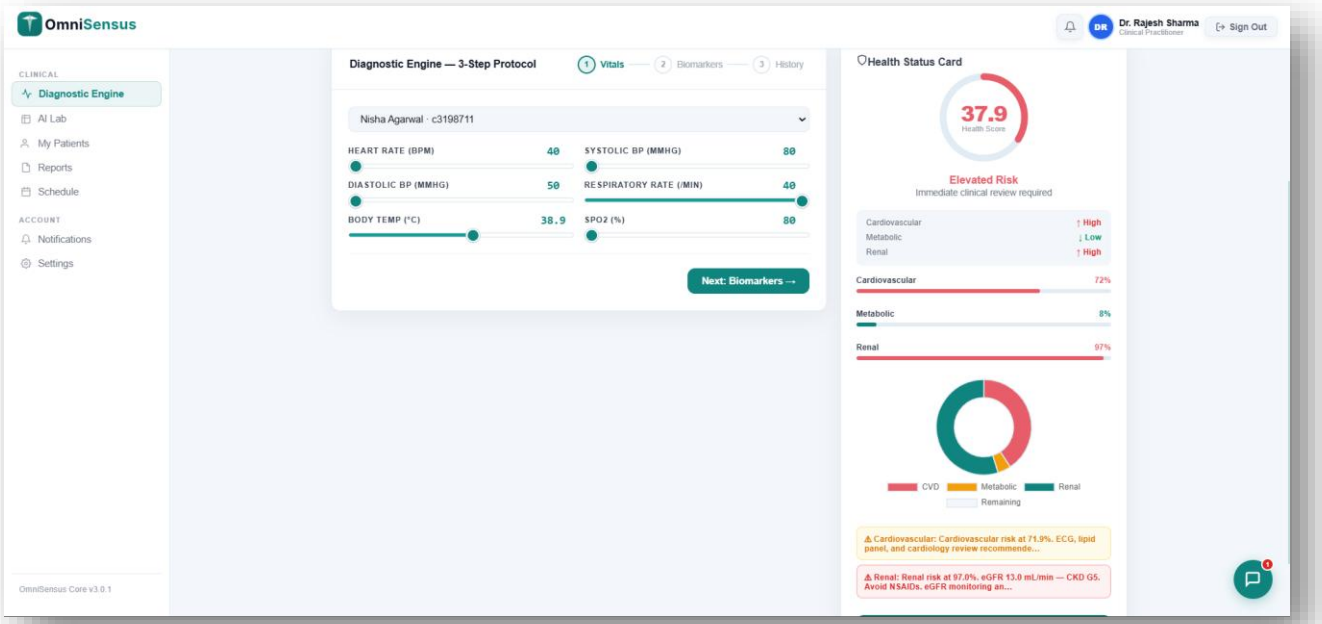


Doctor Diagnostic Wizard

6.2.5 ML Diagnostic Results

A complete diagnostic result demonstrates the system output: Composite Health Score 56.6/100 → Borderline tier; Domain Scores: Cardiovascular 69.7, Metabolic 31.5, Renal 96.5; Clinical Flags: HbA1c 7.0% (borderline), eGFR 11.7 mL/min — CKD Stage 4 (critical); Top SHAP Features: hba1c (0.3789), glucose (0.2686), creatinine (0.2153), hdl (0.1391), triglycerides (0.1348); Triage Urgency: Semi-Urgent.

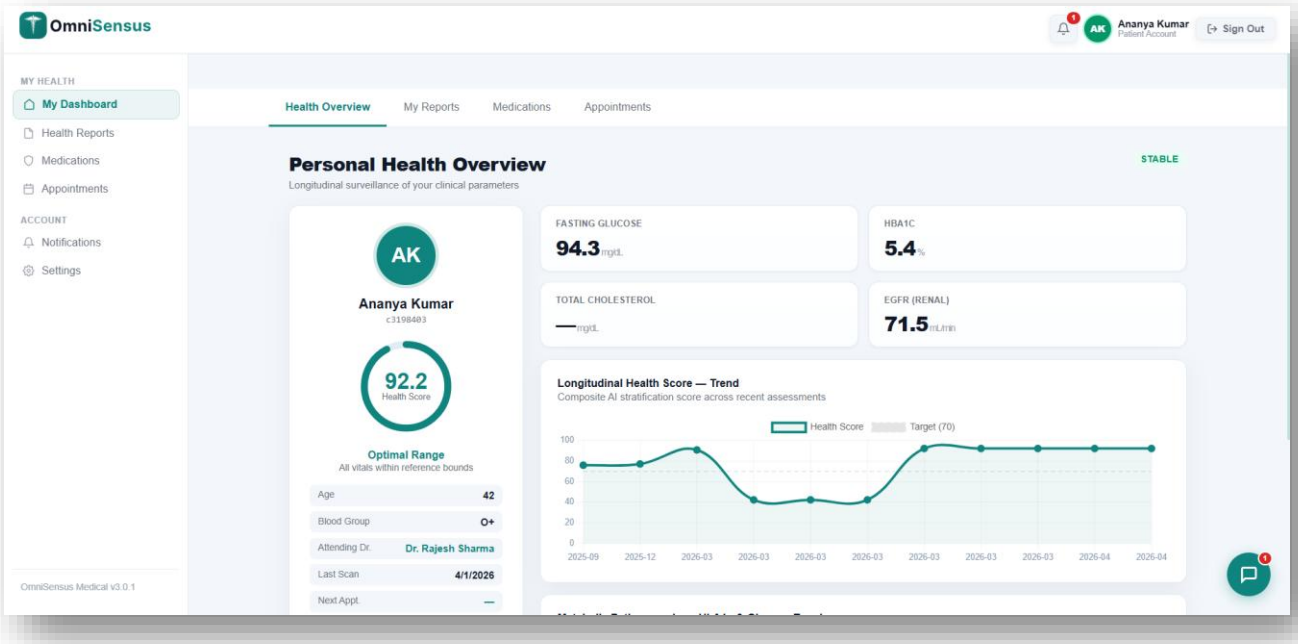


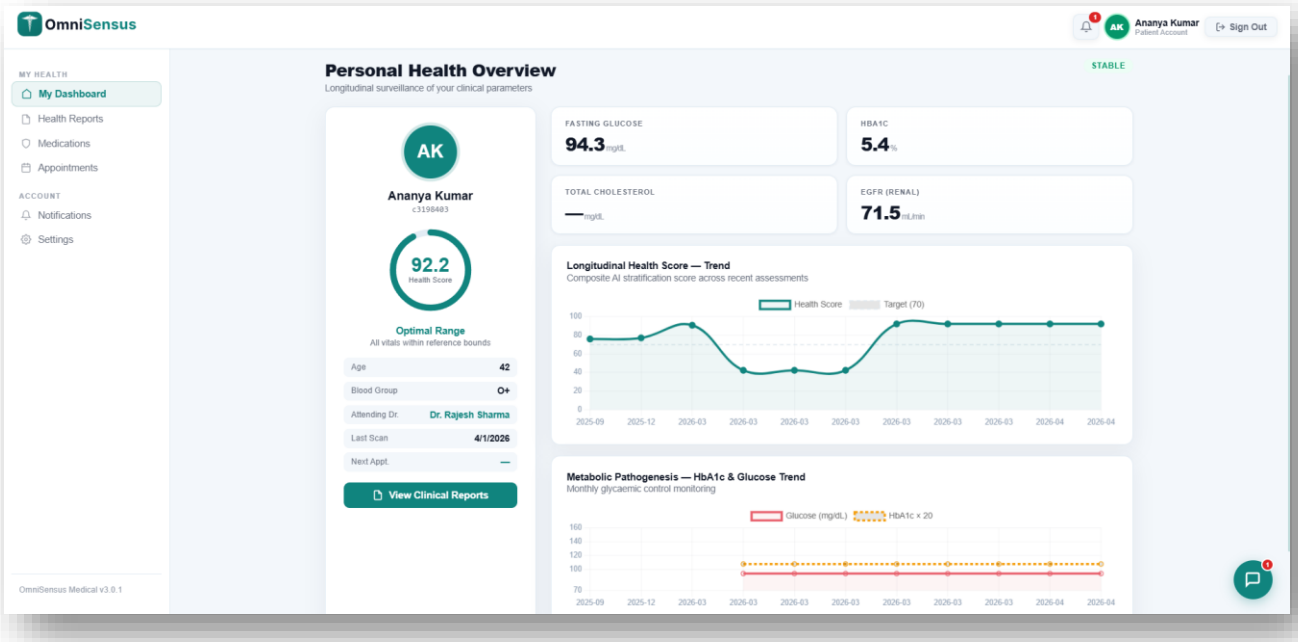


ML Diagnostic Results

6.2.6 Patient Dashboard

The patient dashboard presents health data in an accessible, non-clinical format. The health score ring displays the current composite score with a colour gradient (green for Stable, amber for Borderline, red for Critical). A line chart displays the patient's health score trajectory over their last 6 diagnostic runs.

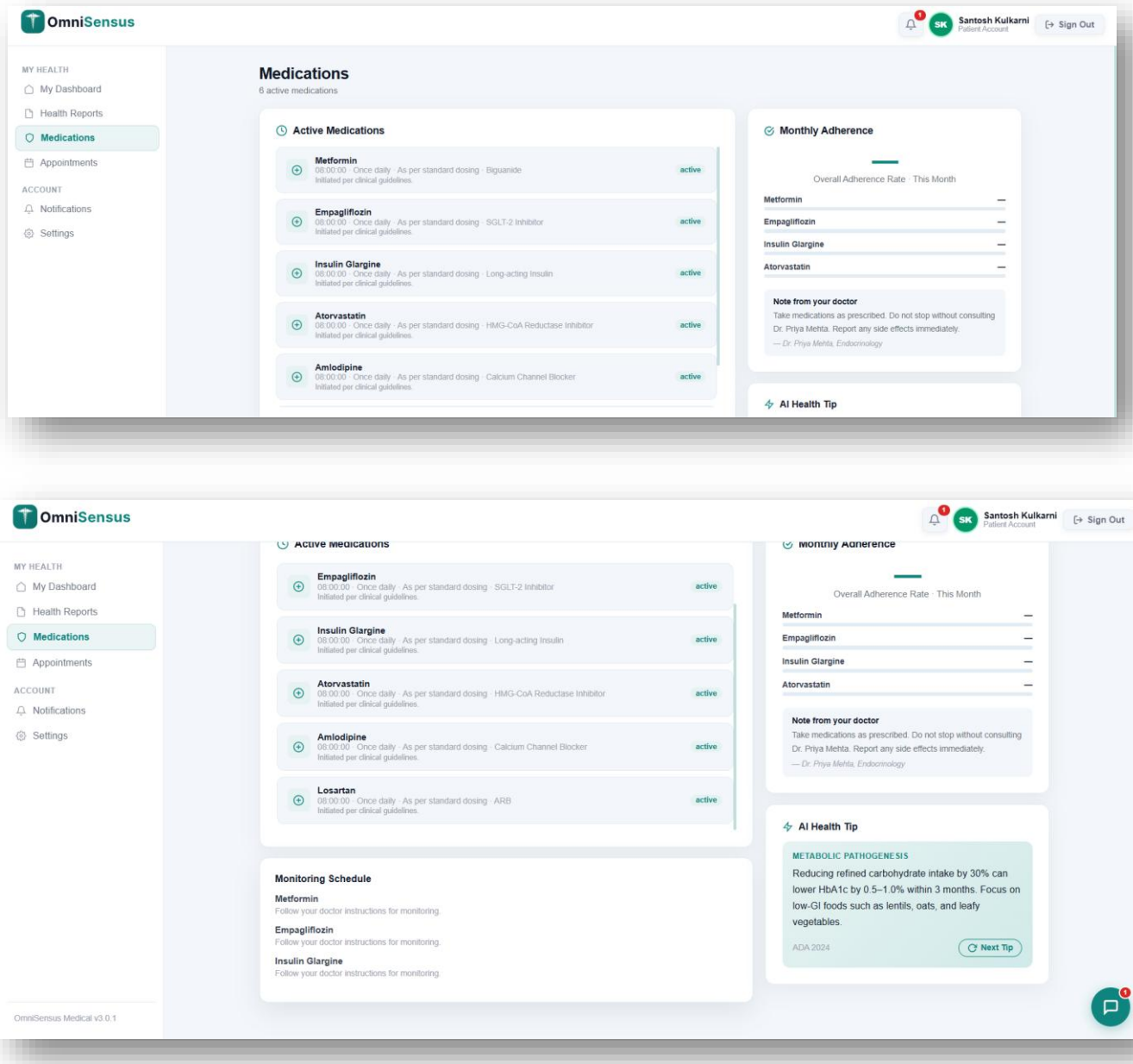




Patient Dashboard

6.2.7 Medications Page

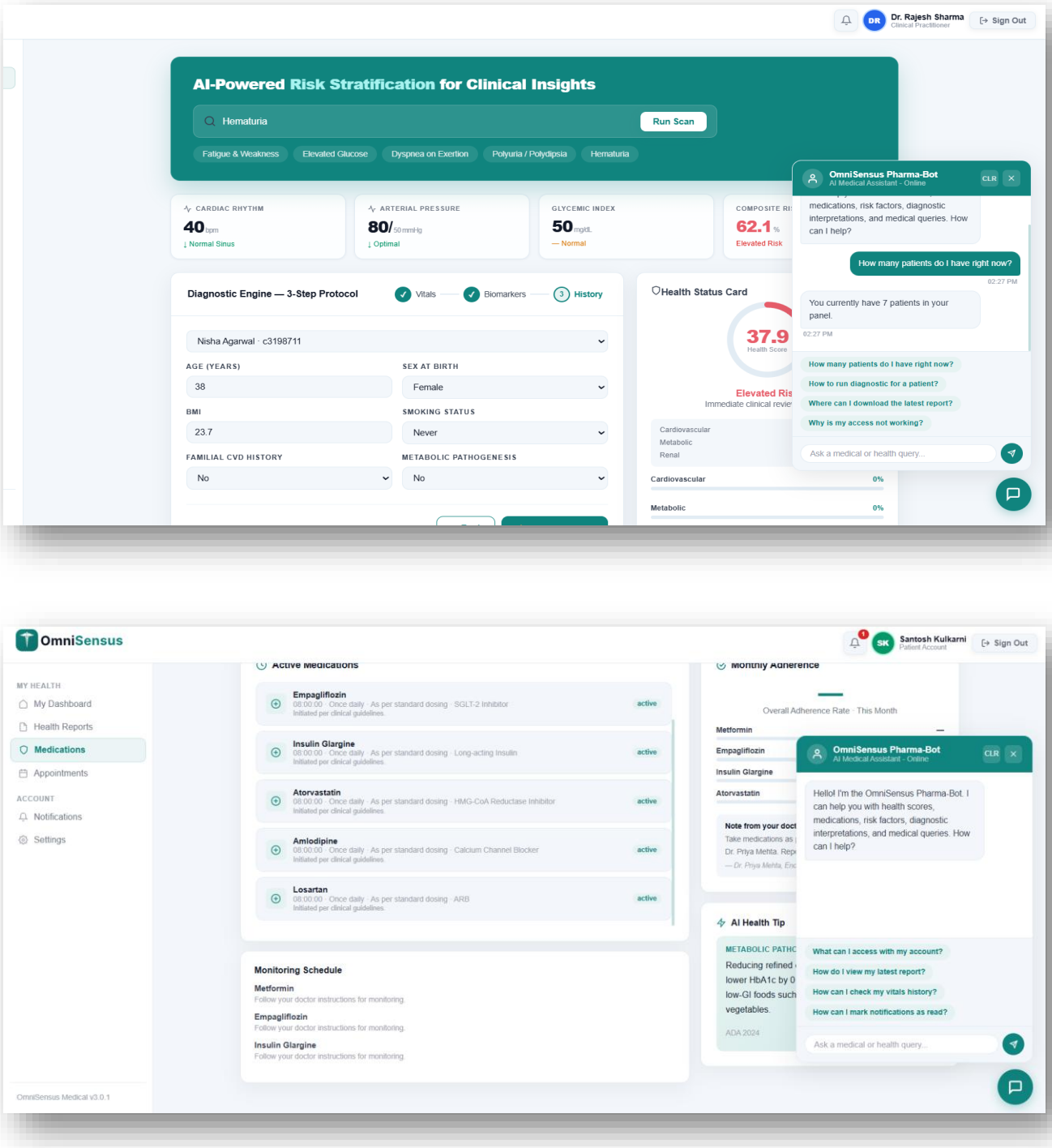
The Patient Medications page displays all active prescriptions from patient_medications JOIN medications. For each medication, the page shows the prescribed dose, frequency, scheduled time, days since last dose, and 30-day adherence percentage computed from the medication_adherence table.



Medications Page

6.2.8 PharmaBot Chat Interface

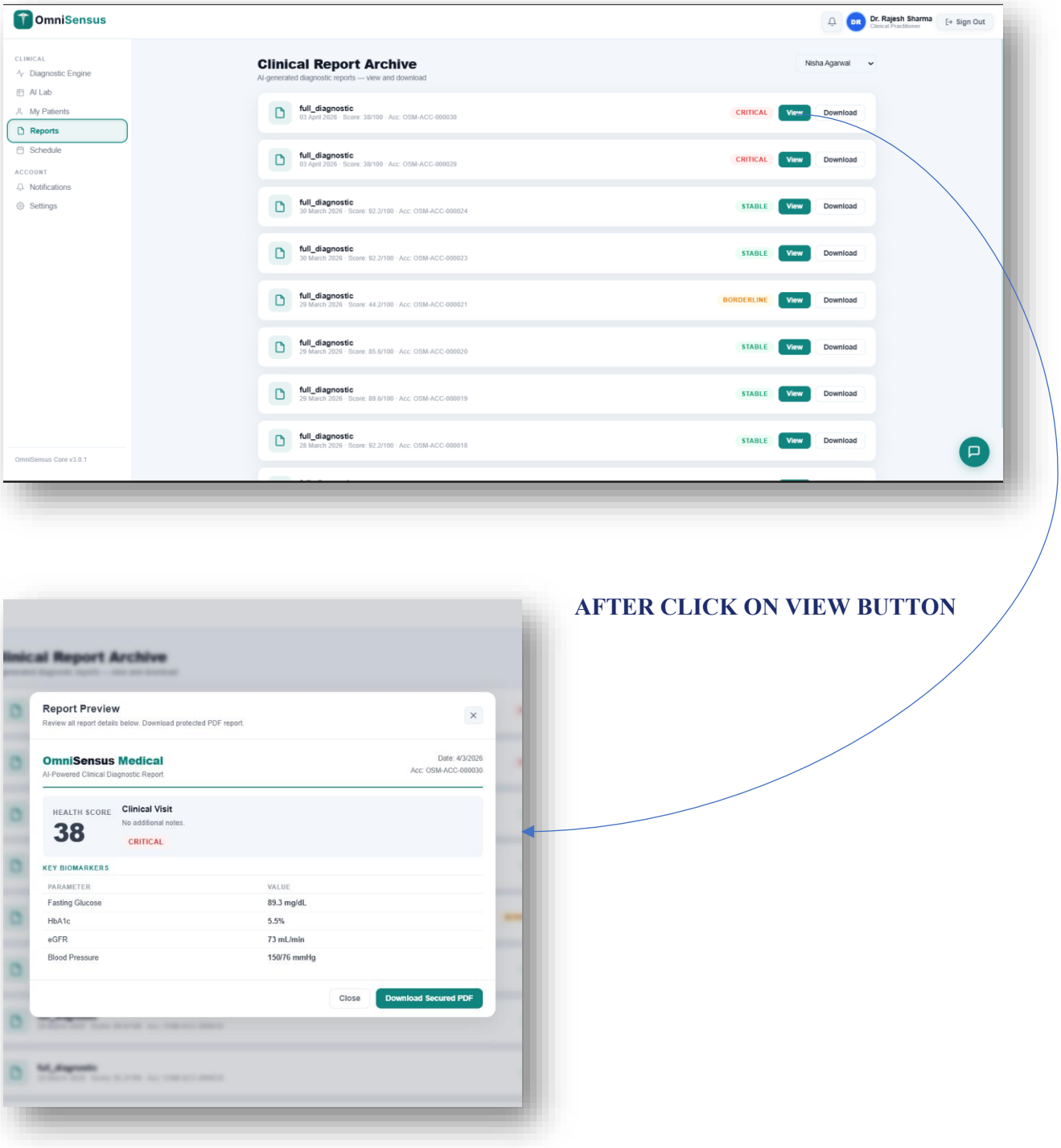
PharmaBot is a conversational AI assistant accessible from all Patient and Doctor pages. User queries are forwarded to the Flask /chat endpoint, which uses the patient's current biomarkers and medication list as context to generate clinically relevant responses. Sessions are persistent across browser refreshes using the chat_sessions and chat_messages tables.



PharmaBot Chat Interface

6.2.9 PDF Diagnostic Reports

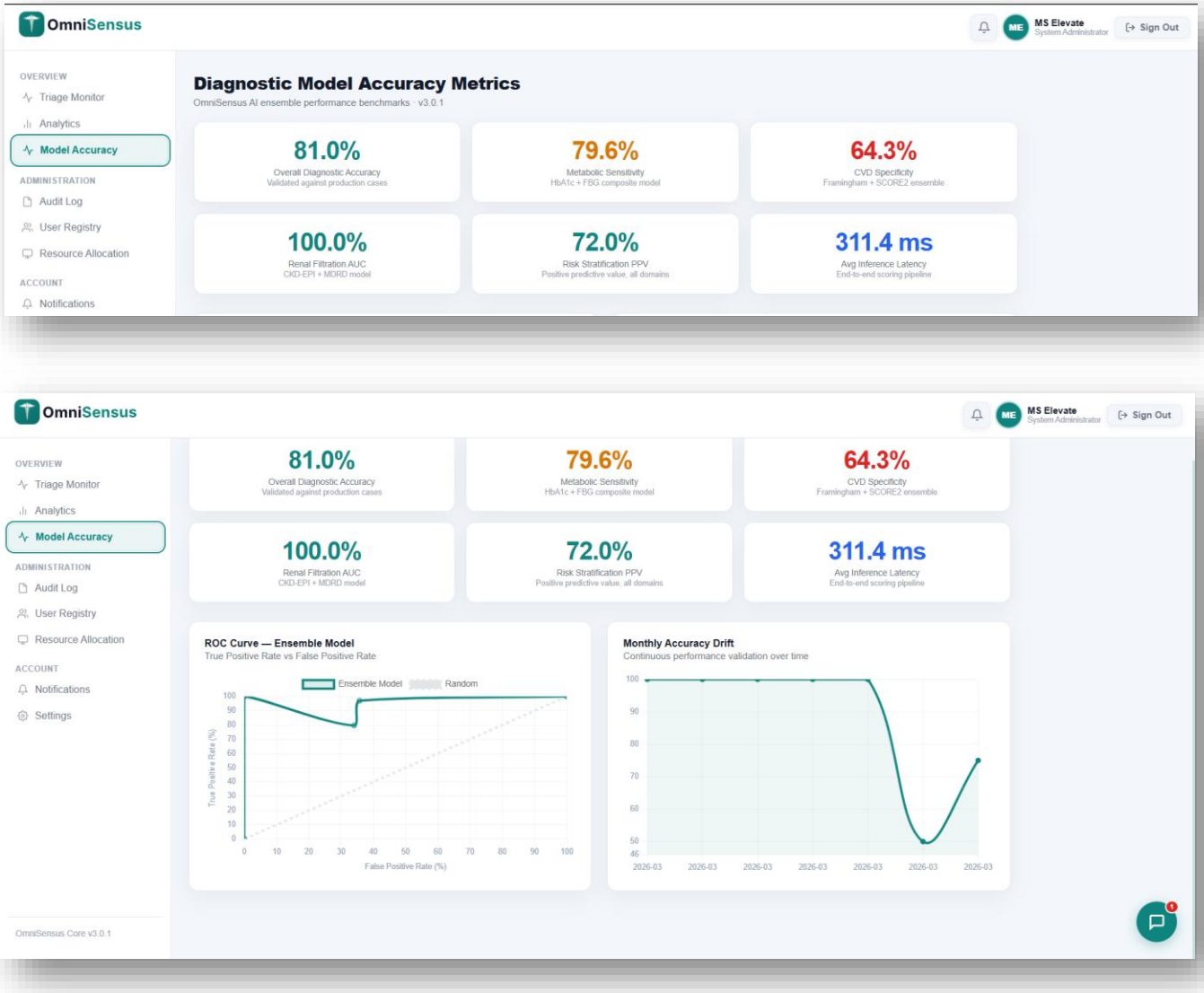
PDF reports are generated by the Flask ML service using ReportLab. Each report includes patient demographics, composite health score with tier classification, domain score breakdown, biomarker table with reference ranges, clinical flags, top-5 SHAP feature importances, triage urgency assessment, and physician notes.



PDF Diagnostic Reports

6.2.10 Admin Model Performance

The Admin Model Accuracy page displays per-model evaluation metrics: ROC-AUC scores (Heart: 91.3%, Diabetes: 82.1%, Kidney: 100%), daily success rate chart from v_model_performance, average inference latency (ms), and timeout/error rates.



Admin Model Performance

CHAPTER 7. PROCESS MODEL

7.1 Agile Process Model

OmniSensus Medical was developed following the Agile Iterative and Incremental methodology. The 12-week development timeline was divided into two-week sprints, each delivering a functional increment of the system. This approach was chosen because the project scope evolved progressively — early sprints established the core authentication and data model, while later sprints layered ML integration, advanced UI components, and v2 database features on top of the stable foundation.

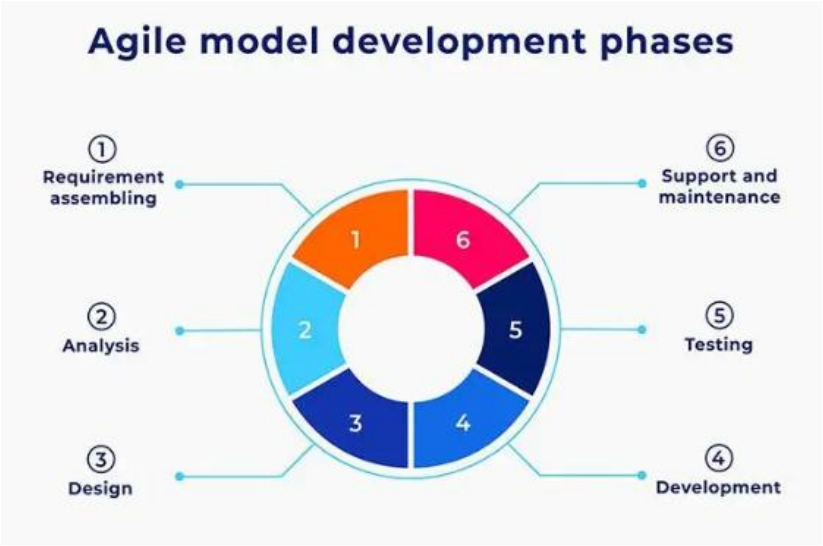


Fig 7.1 : Agile Process Model – Iterative & Incremental

The Iterative and Incremental Model phases applied to this project:

30. Initial Planning: Requirements analysis, technology selection, architecture design, and ER diagram. (Week 1–2)
31. Requirements: Detailed endpoint specification, UI wireframes for all three role dashboards, database table definitions. (Week 2–3)
32. Analysis and Design: System architecture documentation, ML model selection and evaluation, CSS design system colour palette and component library definition. (Week 3–4)
33. Implementation: Backend API development, ML service development, Frontend development — each as parallel workstreams converging at integration milestones. (Week 4–10)
34. Testing: Postman API testing for all 60+ endpoints, frontend functional testing across all three roles, database constraint and trigger verification using seed data. (Week 10–11)

35. Deployment: Render deployment for FastAPI and Flask services, Neon database provisioning, environment variable configuration, and end-to-end integration testing on cloud infrastructure. (Week 11–12)
36. Evaluation: Internal review against project objectives, performance benchmarking (ML inference latency, API response times), and documentation completeness review. (Week 12)
37. Planning (Next Iteration): Identification of future enhancements — deep learning models, Claude API integration, patient appointment booking from frontend, and mobile application development.

CHAPTER 8. CONCLUSION AND DISCUSSION

8.1 Overall Analysis of Internship and Project Viabilities

- Full-Stack Development Exposure: Gained end-to-end experience across the complete web application stack — Next.js 14 frontend, FastAPI backend, Flask ML microservice, and PostgreSQL database — including deployment on cloud infrastructure.
- Machine Learning Integration: Successfully trained, evaluated, and deployed three calibrated Random Forest classifiers within a production web API. Implemented SHAP explainability to make ML predictions interpretable for clinical users.
- Database Engineering: Designed and implemented a 27-table relational schema with complex relationships, automated triggers, and analytical views — skills directly transferable to enterprise software engineering roles.
- API Security: Implemented JWT Bearer authentication, refresh token rotation, bcrypt hashing, role-based access control, SSL database connections, and immutable audit logging.
- Clinical Domain Knowledge: Developed understanding of clinical biomarker ranges, disease risk stratification, medication adherence tracking, and healthcare data privacy considerations.
- Frontend Engineering: Built a 15+ page, 27-CSS-file frontend application with three complete role dashboards, Chart.js data visualisations, and responsive design.
- DevOps and Deployment: Gained practical experience deploying a multi-service application on cloud infrastructure (Render), managing environment variables, configuring CORS, and troubleshooting cold-start latency issues.
- Technical Documentation: Produced comprehensive project documentation including API endpoint references, database ER diagrams, UML diagrams (use case, activity, sequence, class, DFD), ML model specifications, and seed data documentation.
- Problem Solving Under Constraints: Navigated real engineering constraints including Render free-tier cold-start delays (30s+), CORS configuration between three services, and Neon PostgreSQL connection pooling limits.

8.2 Summary of Internship and Project Work

Project Title: OmniSensus Medical — Automated Clinical Decision Support & Integrated Health Intelligence

OmniSensus Medical is a production-grade, full-stack healthcare intelligence platform developed over 12 weeks as part of the Microsoft Elevate internship programme. The system addresses the core problem of fragmented and manually intensive clinical diagnosis by providing an AI-powered, multi-role platform that automates risk stratification for cardiovascular, metabolic, and renal disease domains.

The platform integrates three CalibratedClassifierCV(RandomForest) machine learning models into a unified composite health scoring system. A FastAPI backend orchestrates 60+ REST endpoints, a Flask ML microservice handles inference and PDF generation, and a Next.js 14 frontend delivers three role-specific dashboards (Admin, Doctor, Patient). All data is persisted in a 27-table Neon PostgreSQL database with 8 analytical views and 12 automated triggers.

Key platform achievements include: 67 diagnostic runs across 16 patients with full biomarker, SHAP, and clinical flag records; automated medication adherence tracking at the dose level; a PharmaBot conversational AI assistant; PDF diagnostic report generation; institutional analytics with 15-day trend data; real-time triage monitoring with tier-based prioritisation; and a complete audit logging system with immutable records.

8.3 Limitations and Future Enhancements

Current Limitations:

- Deep Learning: The LSTM component for time-series health score forecasting was deferred due to hardware memory constraints. The architecture is designed to accommodate this addition as a fourth ML endpoint.
- Render Cold Starts: The Render free tier imposes a ~30-second cold-start delay when the service has been idle. A keep-alive ping service partially mitigates this.
- PDF Download: Full streaming download from the ML service to the frontend is pending implementation — currently a toast notification is shown.
- Patient Appointment Booking: The backend endpoint and database schema support this; the frontend booking form for patients is a pending UI implementation.
- Real AI Chat: PharmaBot uses a knowledge-base plus regex fallback. Integration with the Claude API for fully conversational, context-aware clinical AI is the intended production implementation.
- No Multi-Language Support: The current system supports English only. Internationalisation (i18n) support for regional Indian languages (Hindi, Gujarati) would increase accessibility.

Future Enhancements:

- LSTM Time-Series Forecasting: Implement a long short-term memory neural network trained on each patient's longitudinal biomarker data to predict health score trajectories 30, 60, and 90 days forward.
- Claude API Integration: Replace the PharmaBot regex fallback with the Anthropic Claude API, injecting patient biomarker context and medication history into the system prompt for accurate, personalised clinical AI conversations.
- FHIR R4 Compliance: Implement a Fast Healthcare Interoperability Resources (FHIR) R4 API layer to enable interoperability with existing Electronic Health Record (EHR) systems.
- Mobile Application: Develop a React Native cross-platform mobile application for patient-facing features, including push notification medication reminders and biometric authentication.
- Multi-Tenant SaaS: Evolve the platform into a multi-tenant Software-as-a-Service product with hospital-level data isolation, custom subdomain routing, and tiered billing plans.
- Telemedicine Integration: Integrate WebRTC-based video consultation functionality directly within the platform.
- Continuous Model Learning: Implement a model retraining pipeline triggered when the ai_insights data accumulates sufficient new labelled examples, enabling the models to improve over time with real clinical usage data.

REFERENCES

Python and FastAPI:

- https://www.python.org/— Official Python language documentation.
- https://fastapi.tiangolo.com/ — FastAPI framework official documentation.
- https://www.uvicorn.org/ — ASGI server used with FastAPI.

Machine Learning:

- https://scikit-learn.org/ — scikit-learn machine learning library documentation.
- https://shap.readthedocs.io/ — SHAP explainability library documentation.
- https://www.kaggle.com/datasets/andrewmvd/heart-failure-clinical-data — Heart disease dataset reference.
- https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database — PIMA Diabetes dataset reference.
- https://www.kaggle.com/datasets/mansoordaku/ckdisease — CKD dataset reference.

Flask and PDF Generation:

- https://flask.palletsprojects.com/ — Flask web framework documentation.
- https://docs.reportlab.com/ — ReportLab PDF generation documentation.

Next.js and React:

- https://nextjs.org/docs — Next.js 14 App Router documentation.
- https://react.dev/ — React 18 official documentation.
- https://www.chartjs.org/docs/latest/ — Chart.js 4.4 data visualisation documentation.

PostgreSQL and Neon:

- https://www.postgresql.org/docs/ — PostgreSQL 16 official documentation.
- https://neon.tech/docs — Neon Serverless PostgreSQL documentation.

Authentication and Security:

- https://jwt.io/ — JSON Web Token specification and debugging tool.
- https://passlib.readthedocs.io/ — bcrypt password hashing library documentation.

Deployment:

- https://render.com/docs — Render cloud deployment platform documentation.
- https://netlify.com/docs — Netlify Next.js deployment documentation.

Development Tools:

- https://code.visualstudio.com/ — Visual Studio Code editor.
- https://www.postman.com/ — Postman API testing platform.
- https://github.com/ — GitHub version control and repository hosting.
- https://stackoverflow.com/ — Developer community support.

Healthcare Standards:

- American Diabetes Association (ADA). Standards of Medical Care in Diabetes — 2024. Diabetes Care, 47(Supplement 1).
- KDIGO 2024 Clinical Practice Guideline for Chronic Kidney Disease.
- AHA/ACC 2023 Guideline for the Management of Heart Failure.
- ESC 2023 Guidelines for the Management of Arterial Hypertension.